CSE344 Software Engineering (SWE)

Week – 1 Introduction

Innovative Modeling Approaches for Production Systems to Raise Validatable Efficiency

Historical Definition

- The term *Software Engineering* first introduced in the first NATO conference in 1968.
- It was *defined then* as follows [2]:
  - The establishment and use of *sound* engineering principles in order to *economically* obtain *reliable* software that works *efficiently* on *real machines*.

About Historical Definition

- The definition was very modern since it is still valid.
- Software engineering
  - is *disciplined* engineering work,
  - offers *means to* build *high-quality*, *efficient* software at *affordable prices*, and
  - offers *task allocation* and *tools* for all software building phases.

Preliminary Questions on SWE

- What is software?
- What is software engineering?
- What is the difference between software engineering and computer science?
- What is the difference between software engineering and system engineering?
- What is a software process?
- What is a software process model?
- What are the costs of software engineering?
- What are software engineering methods?
- What is CASE (Computer-Aided Software Engineering)
- What are the attributes of good software?
- What are the key challenges facing software engineering?

What is SW?

- SW is an *interconnected set of computer progams and associated documentation* such as
  - *data* and *configuration* files, *requirements*, *design models*, *test plans*, *cases* and *results*, *user and technical manuals*
- Two types of SW products
  - *generic*: developed to sell to multiple customers
  - *bespoke* or *custom*: developed for a single customer

What is SWE?

- SWE is an *engineering discipline* concerned with all aspects of *software production*.
- Software engineers should
  - adopt a *systematic and organised approach* to their work, and
  - use *appropriate tools and techniques* depending on
    - the problem to be solved,
    - the development constraints and
    - the resources available.

What is the difference between SWE and computer science (CS)?

- CS ↔ *theory and fundamentals*
- SWE ↔ *practicalities of developing and delivering useful software*.
- CS theories still insufficient as a complete foundation for SWE unlike e.g. physics and electrical engineering.

What is the difference between SWE and system engineering (SyE)?

- *A computer-based system* ↔ a set of interrelated components
  - *hardware* and *software* components,
  - users and use environments.
- That is, *SW components are a part of the computer-based system*.
- SyE ↔ computer-based systems development including
  - hardware,
  - software and
  - process engineering.
- SWE is part of this process concerned with developing
  - the SW infrastructure,
  - control,
  - applications and
  - databases in the system.

What is a SW process?

- A set of activities to develop/evolve SW.
- *Generic activities* in all SW processes are:
  - ***Specification*** - what the system should do and its development constraints
  - ***Development*** – how to produce the SW system
  - ***Validation & Verification (V&V)*** - checking that
    - the SW product is what the customer really wants, and
    - the SW product does what it is supposed to.
  - ***Evolution*** - changing the software in response to changing demands.

What is a SW process model?

- A *simplified representation* of a SW process
  - presented *from a specific perspective*.
- Examples of *process perspectives*
  - *Workflow perspective* - sequence of activities;
  - *Data-flow perspective* - information flow;
  - *Role/action perspective* - who does what.
- ***Generic process models***
  - ***Waterfall***;
  - ***Iterative development***;
  - ***Component-based software engineering***.

What are the costs of SWE?

- Roughly,
  - ***60% ↔ development costs***,
  - ***40% ↔ testing costs***.
- For ***custom SW***, ***evolution*** *(or* ***maintenance****)* ***costs*** *often exceed development costs*.
- Costs vary depending on
  - the *type of system being developed* and
  - the *requirements of system attributes* such as *performance and system reliability*.
- Distribution of costs depends on *process model* used.

What are SWE methods?

- *Structured approaches to SW development* such as
  - system models, notations, rules,
  - design advice and process guidance.
- *Model descriptions*
  - Descriptions of graphical models. They *should be* produced;
- *Rules*
  - Constraints applied to system models;
- *Recommendations*
  - Advice on good design practice;
- *Process guidance*
  - What activities to follow.

What is CASE (Computer-Aided Software Engineering)?

- Software systems that *provide automated support* for SW process activities.
- CASE systems are often used for *method support*.
- ***Upper-CASE***
  - Tools to support the early process activities of *requirements and design*;
- ***Lower-CASE***
  - Tools to support later activities such as *programming, debugging and testing*.

What are the attributes of good SW?

- SW should deliver the *required functionality* and *performance* to the user and should be *maintainable, dependable and acceptable*.
- ***Maintainability***
  - Software must evolve (i.e., be changed) to meet changing needs;
- ***Dependability (Reliability-Safety-Security)***
  - Software must be trustworthy;
- ***Efficiency***
  - Software should not make wasteful use of system resources;
- ***Acceptability***
  - Software must accepted by the users for which it was designed. This means it must be *understandable, usable and compatible* with other systems.

Supplementary slide – 1... Reliability

Reliability (Güvenilirlik): The ability of the SW product to function within a prespecified span of time to fulfill a specific purpose under certain conditions as expected.

Supplementary slide – 2... Safety

Safety (Güvenlik): The capability of the SW product to end the functioning of the system it controls/manages without ever damaging the system's environment even if the SW product itself fails.

Supplementary slide – 3... Security

Security (Güvenlik): The ability of the SW product to protect itself (and so the system it controls) from any external attacks or unauthorized use/access of system services that be either accidental or deliberate.

What are the key challenges facing SWE?

- *Heterogeneity, delivery and trust*.
- ***Heterogeneity***
  - Developing techniques for building software that can cope with heterogeneous platforms and execution environments;
- ***Delivery***
  - Developing techniques that lead to faster delivery of SW;
- ***Trust***
  - Developing techniques that demonstrate that SW can be trusted by its users.

A Brief History of SWE

1950's... First Commercial SW

- First commercial SW appeared in 1951 in England, made by the company J. Lyons.
- Then,
  - developing SW was
    - *straightforward and*
    - *uncontrolled*.
  - SW could be relatively *versatile* (i.e., adaptable, multi-purpose).

Early 1960's... SW Crisis

- Early 1960's... First appearance of terms such as
  - *testability*
  - *interfaces*.
- *SW development still not structured*.
- Average size of SW grew and problems got worse.
- Projects started to miss their budget and schedule.
- Unstructured SW was not good enough any more.
- ***The SW crisis of 1960's!!!***

Late 1960's... First SWE Conferences

- 1968... NATO organized the first SWE conference
- The term "***Software Engineering***" was introduced.
- 1969... The second SWE conference by NATO.
- The *basis of "Software Engineering"* was defined in these two conferences.

Results of the first Conferences

1. These conferences did not solve the SW crisis.
2. The crisis seems not to be solved even today. (Why? How to see this?)
3. However, ideas emerged then brought about a solid foundation for "*modern Software Engineering*."

1970's... SW Process Models

- Early 1970's... The first *SW process model* ***waterfall*** model was defined.
- The *waterfall* model defined a *systems engineering standard* and it was originally used for *hardware (and other) engineering* fields.

1970's and 1980's...More on Process Models

- The systems engineering model was good enough for other engineering branches, but not for SWE. (Why?) Most importantly product intangibility
- Thus, in 70's and 80's several *new software process models* were introduced

Programming Languages

- 1950's... Machine language was employed to develop the first programs, but
- *first* high-level programming languages were already known at...
  - Early 1960's... ***Cobol and PL/1*** were introduced.
  - ***Late 1960's ...*** *Introduction of* ***C***.

1980's... Object-Oriented Programming (OOP)

- 1980's... *OOP* is proposed.
- 1986... The *first conference OOPSLA* on OOPs.
- OO philosophy was long since discussed, but the first OOP languages started to solidly use the principles.
- OO philosophy spread slowly from languages to all SWE fields.

Personal Computing

- Early 1990's... SWE quite stable...
- The term ***personal computing*** emerged and changed needs for SW but not the techniques of building SW.
- *Computer-Aided Software Engineering (CASE)* tools or ***CASE-tools*** appeared and evolved fast.

1999... Agile Process Models

- 1999... Introduction of ***Extreme Programming (XP)***
- *XP*, an *agile process model,* was the most recent change in SWE
- Agile process models soon became popular among programmers and small/medium companies because XP is a suitable model to use to develop small/medium-scale SW projects for both design and programming issues.

Current... Distributed SW systems

- Currently, the term ***distributed SW architecture*** is an essential concept to explain today's SW products.
- Although the basic principles of SWE have not changed much, companies develop their own way of handling SW process. They use their ***specialized*** process models, methods and tools.

Current... Specialized SW Products

- SW is specialized from company to company.
- For any-scale SW products ranging from
  - *small-scale SW* (e.g., cell phone SW and other mobile equipment) to
  - *large-scale SW* (e.g., jumbo airplane (BOEING) navigation systems, complex weapon control systems, nuclear reactor systems),
- the *need for specialization is to remain in the market and keep or enhance their share in the market*.

Future...?

- Three potential challenges in the future:
  - Heterogeneity challenge.
  - Delivery challenge.
  - Trust challenge.
- They are not independent; may even conflict with each other.

Heterogeneity Challenge

- Software-based systems have to work in *various environments cooperating with different types of systems*.
- Software has to be built to *work both with current and legacy (old) systems*.
- Legacy systems often need new functionality.
- The life span of old SW is often expanded beyond natural limits.

Delivery Challenge

- SW development takes *unexpectedly much time*. Time is an expensive resource.
- Business is an extremely dynamic domain and requires any system it involves *be ready for changes*.
- So, fast development of maintainable/evolvable SW is inevitable and *the maintenance to SW should not compromise from quality*.

Trust Challenge

- Software systems affect more and more our lives.
- The more they control, the more we need to trust them.
- *Trust does not come easily. One has to be able to see that software works as planned*.
- Currently such *negative phenomena such as spam, viruses and worms damage trust*.

References

[1] Ian Sommerville, Software Engineering, 8th ed. 2007

[2] P. Naur, R.Randell (eds.): Software Engineering: A Report on a Conference Sponsored by the NATO Science Committee, 1968

CSE344 Software Engineering (SWE)

Week – 2 SW Processes

What is a SW process?

- A set of activities to develop/evolve SW.
- *Generic activities* in all SW processes are:
  - ***Specification*** - ***what*** the system should do and its development constraints
  - ***Development*** – ***how*** to produce the SW system
  - ***Validation & Verification (V&V)*** - checking that
    - the SW product is what the customer really wants, and
    - the SW product does what it is supposed to.
    - two questions symbolizing V&V:
      - ***Am I doing the right product?*** ↔ ***Validation***
      - ***Am I doing the product right?*** ↔ ***Verification***
  - ***Evolution*** - changing the software in response to changing demands.

SW Process Model...

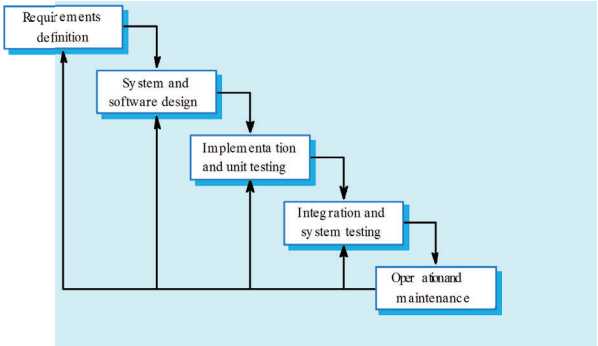
Definition:

A ***SW process model*** is an abstract representation of a process. It presents a description of a process from some particular perspective.

Generic SW Process models

- Waterfall:
  - Separate and distinct phases of specification and development.
- Evolutionary development
  - Specification, development and validation are interleaved.
- Component-based software engineering
  - The system is assembled from existing components.
- Variants of these models e.g., formal development:
  - a waterfall-like process is used
  - a formal specification refined through several stages to an implementable design.

Waterfall model



Waterfall model phases

- Requirements analysis and definition
- System and software design
- Implementation and unit testing
- Integration and system testing
- Operation and maintenance
- Main drawback:
  - Inflexible to change after the process is underway. One phase has to be complete before moving onto the next phase.

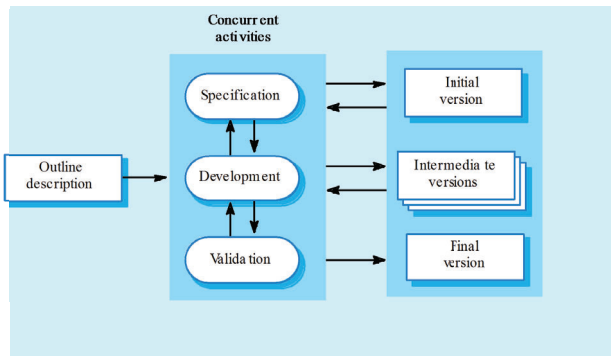
Waterfall model problems

- Inflexible partitioning of process into stages: Hard to handle changing customer requirements.
- Hence, waterfall is appropriate for projects with stable and well-understood requirements and hence, with limited changes during the design process.
- Few business systems have stable requirements.
- The waterfall model is mostly used for large systems engineering projects where a system is developed at several sites.

Evolutionary development

- Exploratory development
  - Objective is to work with customers and *to evolve a final system* from an initial outline specification. *Should start with well-understood requirements* and add new features as proposed by the customer.
- Throw-away prototyping
  - Objective is to *understand the system requirements*. *Should start with poorly understood requirements* to clarify what is really needed.

Evolutionary development



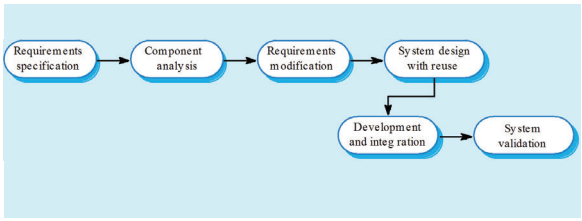
Evolutionary development

- ***Problems***
  - *Lack of process visibility*;
  - *Systems are often poorly structured*;
  - *Special skills* (*e.g. in languages for rapid prototyping*) may be required.
- ***Applicability***
  - *For small or medium-size interactive systems*;
  - *For parts of large systems (e.g. the user interface)*;
  - *For short-lifetime systems*.

Component-based software engineering

- Based on ***systematic reuse*** where systems are integrated from ***existing components*** or COTS (***Commercial-off-the-shelf***) systems.
- ***Process stages***
  - *Component analysis*;
  - *Requirements modification*;
  - *System design with reuse*;
  - *Development and integration*.
- This approach is ***becoming increasingly used*** as component standards have emerged.

Reuse-oriented development



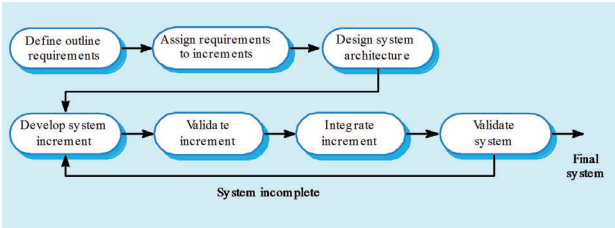
Process iteration

- System *requirements always evolve* during a project so process iteration where *earlier stages are reworked* is always part of the process for large systems.
- *Iteration can be applied to any of the generic process models*.
- Two (related) approaches
  - *Incremental delivery*;
  - *Spiral development*.

Incremental delivery

- Not a single delivery;
- Development and delivery in increments
- Each increment satisfies part of the required functionality.
- User requirements are prioritised and the higher the priority of a requirement the sooner it is delivered.
- Current increment's requirements are frozen; but requirements for later increments may evolve.

Incremental development



Incremental development advantages

- Delivery of increments provides *earlier availability of the system functionality*.
- *Early increments* act as a prototype to help *elicit requirements for later increments*.
- *Lower risk of overall project failure*.
- The *highest priority system services* tend to receive the *most testing*.

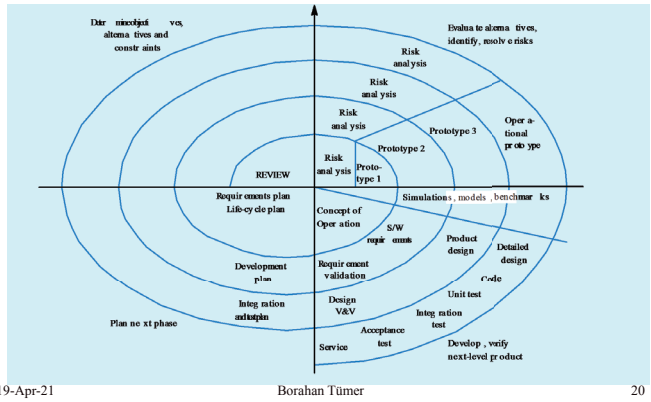
Extreme programming

- A derivative of incremental development.
- An approach to development based on the development and delivery of *very small (even weekly or biweekly) increments of functionality*.
- Relies on
  - *constant code improvement,*
  - *user involvement in the development team* and
  - *pairwise programming*.

Spiral development

- *Spiral representation of the process* rather than as a sequence of activities with backtracking.
- *Each loop in the spiral represents a phase* in the process.
- ***No fixed phases*** such as specification or design - loops in the spiral are chosen depending on what is required.
- ***Risks are explicitly assessed and resolved throughout the process***.

Spiral model of the software process



Spiral model sectors

- *Objective setting*
  - Specific objectives for the phase are identified.
- *Risk assessment and reduction*
  - Risks are assessed and activities put in place to reduce the key risks.
- *Development and validation*
  - A development model for the system is chosen which can be any of the generic models.
- *Planning*
  - The project is reviewed and the next phase of the spiral is planned.

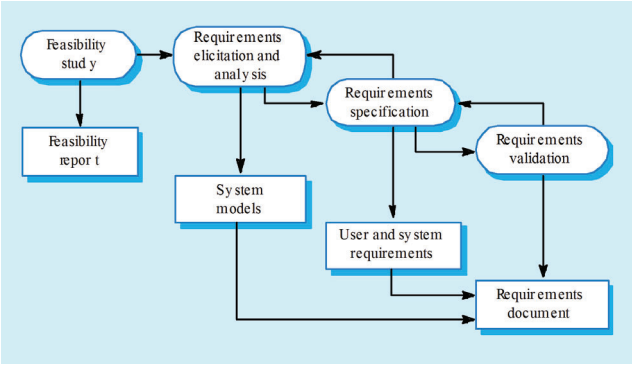
Process activities

- *Software specification*
- *Software design and implementation*
- *Software validation & verification*
- *Software evolution*

Software specification

- The *process of establishing what services are required and the constraints on the system's operation and development*.
- ***Requirements engineering*** process
  - *Feasibility study*;
  - *Requirements elicitation and analysis*;
  - *Requirements specification*;
  - *Requirements validation*.

The requirements engineering process



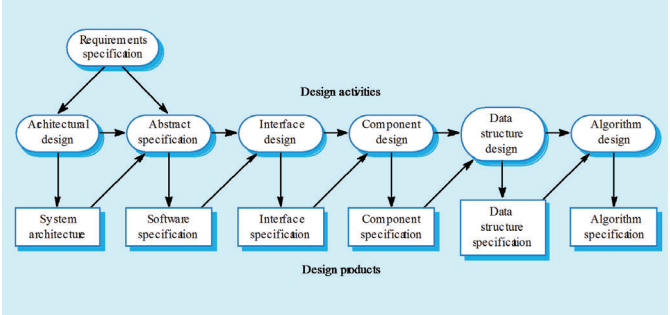
Software design and implementation

- The process of *converting the system specification into an executable system*.
- *Software design*
  - Design a software structure that realises the specification;
- *Implementation*
  - Translation of design to an executable program;
- Design and implementation may be inter-leaved.

Design process activities

- Architectural design
- Abstract specification
- Interface design
- Component design
- Data structure design
- Algorithm design

The software design process



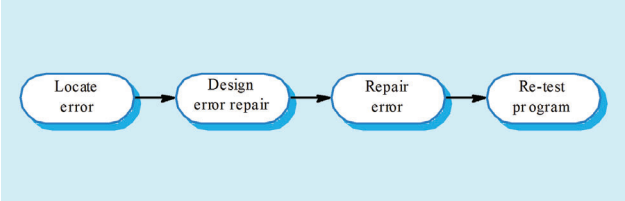
Structured methods

- Systematic approaches to developing a software design.
- The design is usually documented as a set of graphical models.
- Possible models
  - Object model;
  - Sequence model;
  - State transition model;
  - Structural model;
  - Data-flow model.

Programming and debugging

- Design ⇒ program + error removal.
- Programming is a personal activity - there is no generic programming process.
- Programmers carry out some program testing to discover faults in the program and remove these faults in the debugging process.

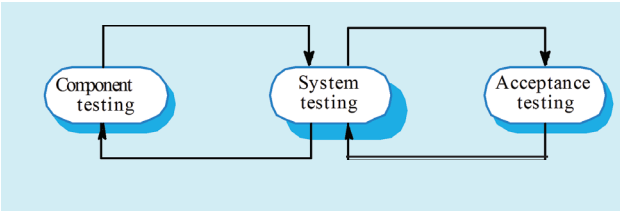
The debugging process



Software validation & verification

- Verification and validation (V & V) is intended to show that a system conforms to its specification and meets the requirements of the system customer.
- Involves checking and review processes and system testing.
- System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system.

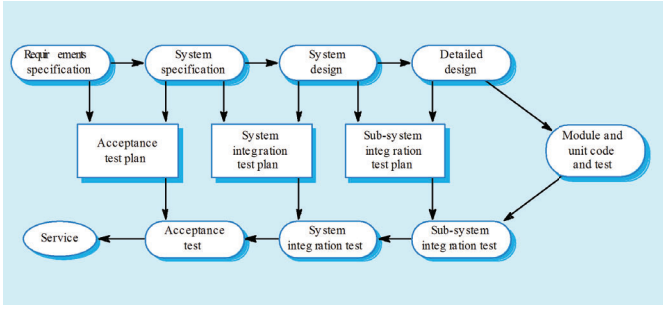
The testing process



Testing stages

- *Component or unit testing*
  - Individual components are tested independently;
  - Components may be functions or objects or coherent groupings of these entities.
- *System testing*
  - Testing of the system as a whole. Testing of emergent properties is particularly important.
- *Acceptance testing*
  - Testing with customer data to check that the system meets the customer's needs.

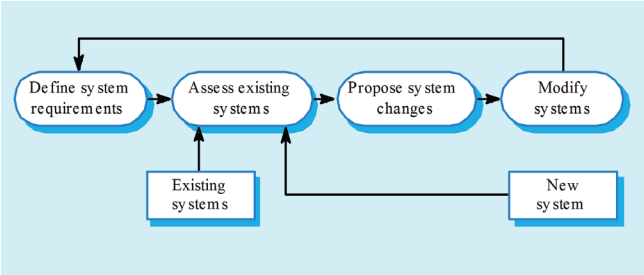
Testing phases



Software evolution

- *Software* is inherently flexible and *can change*.
- As *requirements change through changing business circumstances*, the software that supports the business must also evolve and change.
- Although there has been a isolation between development and evolution (maintenance) this is increasingly irrelevant as fewer and fewer systems are completely new.

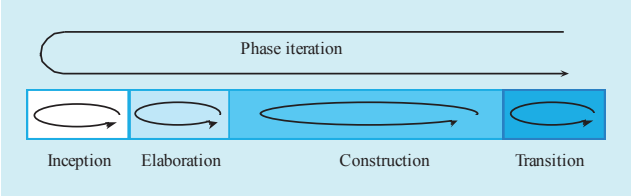
System evolution



The Rational Unified Process

- A modern process model derived from the work on the UML and associated process.
- Normally described from 3 perspectives
  - A *dynamic perspective* that shows phases over time;
  - A *static perspective* that shows process activities;
  - A *practice perspective* that suggests good practice.

RUP phase model

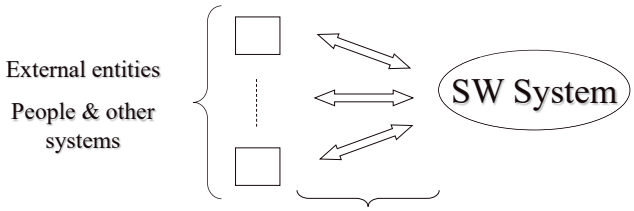


RUP phases

- *Inception*
  - Establish the business case for the system.
- *Elaboration*
  - Develop an understanding of the problem domain and the system architecture.
- *Construction*
  - System design, programming and testing.
- *Transition*
  - Deploy the system in its operating environment.

Inception

• Identify all external entities and their interactions with the SW system



Interactions of SW System with external entities

RUP good practice

- Develop software iteratively
- Manage requirements
- Use component-based architectures
- Visually model software
- Verify software quality
- Control changes to software

Static workflows

Workflow	Description
Business modelling	The business processes are modelled using business use cases.
Requirements	Actors who interact with the system are identified and use cases are developed to model the system requirements.
Analysis and design	A design model is created and documented using architectural models, component models, object models and sequence models.
Implementation	The components in the system are implemented and structured into implementation sub-systems. Automatic code generation from design models helps accelerate this process.
Test	Testing is an iterative process that is carried out in conjunction with implementation. System testing follows the completion of the implementation.
Deployment	A product release is created, distributed to users and installed in their workplace.
Configuration and change management	This supporting workflow managed changes to the system (see Chapter 29).
Project management	This supporting workflow manages the system development (see Chapter 5).
Environment	This workflow is concerned with making appropriate software tools available to the software development team.

Computer-aided software engineering (CASE)

- ... denotes software to support software development and evolution processes.
- ... automates process activities
  - *Graphical editors* for system model development;
  - *Data dictionary* to manage design entities;
  - *Graphical UI builder* for user interface construction;
  - *Debuggers* to support program fault finding;
  - *Automated translators* to generate new versions of a program.

CASE technology

- ... has led to significant improvements in software process, but not the order-of-magnitude (i.e.,essential) improvements as once predicted
  - Software engineering requires *creative thought* - this is not readily automated;
  - Software engineering is a *team activity* and, for large projects, *much time is spent in team interaction*s. CASE technology does not really support these.

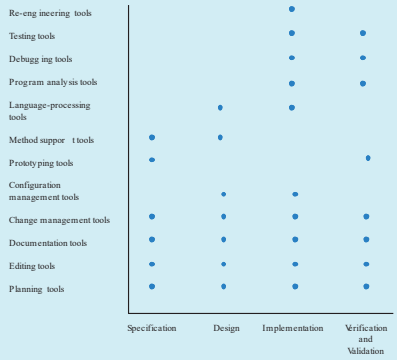
CASE classification

- Classification helps us understand the different types of CASE tools and their support for process activities.
- *Functional perspective*
  - Tools are classified according to their specific function.
- *Process perspective*
  - Tools are classified according to process activities that are supported.
- *Integration perspective*
  - Tools are classified according to their organisation into integrated units.

Functional tool classification

Tool type	Examples
Planning tools	PERT tools, estimation tools, spreadsheets
Editing tools	Text editors, diagram editors, word processors
Change management tools	Requirements traceability tools, change control systems
Configuration management tools	Version management systems, system building tools
Prototyping tools	Very high-level languages, user interface generators
Method-support tools	Design editors, data dictionaries, code generators
Language-processing tools	Compilers, interpreters
Program analysis tools	Cross reference generators, static analysers, dynamic analysers
Testing tools	Test data generators, file comparators
Debugging tools	Interactive debugging systems
Documentation tools	Page layout programs, image editors
Re-engineering tools	Cross-reference systems, program re-structuring systems

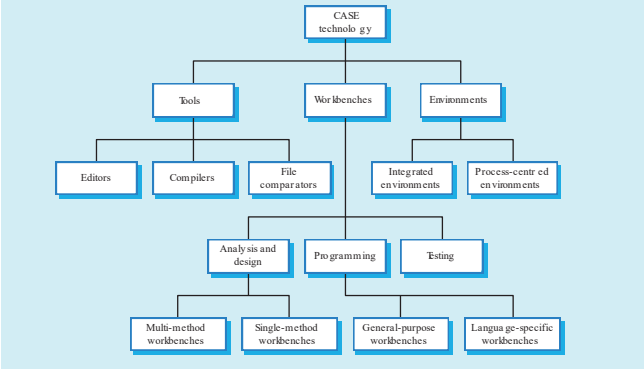
Activity-based tool classification



CASE integration

- Tools
  - Support individual process tasks such as design consistency checking, text editing, etc.
- Workbenches
  - Support a process phase such as specification or design, Normally include a number of integrated tools.
- Environments
  - Support all or a substantial part of an entire software process. Normally include several integrated workbenches.

Tools, workbenches, environments



References

[1] Ian Sommerville, Software Engineering, 8th ed. 2007

[2] P. Naur, R.Randell (eds.): Software Engineering: A Report on a Conference Sponsored by the NATO Science Committee, 1968

CSE344 Software Engineering (SWE)

Week – 3
Requirements
&
Req. Engineering

Requirement (İster) ...

- ***...is a way of expressing a client's need*** (a service or a constraint on the system) ranging at one end from
  - *a high-level abstract statement* at the other end to
  - *a detailed mathematical functional specification*.
- ***...may serve a dual function***
  - *Basis for a bid (öneri) for a contract* ⇔ ***open to interpretation***
  - *Basis for the contract itself* ⇔ ***defined in detail*;**

A Classification of req.s by their reader

- ***User requirements*** (written ***for customers***)
  - *Statements in natural language plus diagrams of the services of the system and its operational constraints.*
- ***System requirements***
  - a *structured* document
  - detailed descriptions of the system's functions, and operational constraints (functionality of the system).
  - *may be part of a contract between client and contractor.*

Definitions and specifications

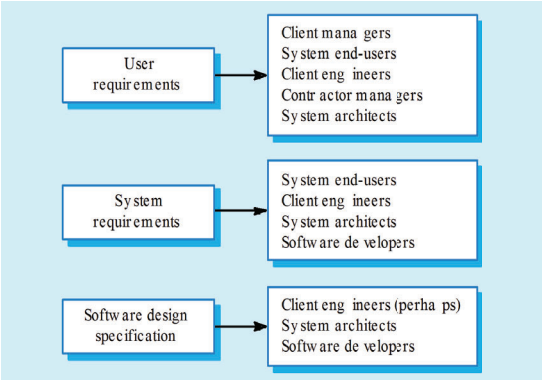
User requirement definition

1. The software must provide a means of representing and accessing external files created by other tools.

System requirements specification

1.1 The user should be provided with facilities to define the type of external files.
1.2 Each external file type may have an associated tool which may be applied to the file.
1.3 Each external file type may be represented as a specific icon on the user's display.
1.4 Facilities should be provided for the icon representing an external file type to be defined by the user.
1.5 When a user selects an icon representing an external file, the effect of that selection is to apply the tool associated with the type of the external file to the file represented by the selected icon.

Requirements readers



Another classification regarding what they state...

- ***Functional requirements***
  - *Statements of services* the system should provide, how the system should react to particular inputs and how the system should behave in particular situations.
- ***Non-functional requirements***
  - *constraints on the services or functions* offered by the system such as timing constraints, constraints on the development process, standards, etc.
- ***Domain requirements***
  - *Requirements that come from the application domain* of the system and that *reflect characteristics of that domain*.

The LIBSYS system

- A library system that provides a single interface to a number of databases of articles in different libraries.
- Users can search for, download and print these articles for personal study.

Examples of functional requirements

1. The user shall be able to search either all of the initial set of databases or select a subset from it.
2. The system shall provide *appropriate viewers* for the user to read documents in the document store.
3. Every order shall be allocated a unique identifier (*ORDER_ID*) which the user shall be able to copy to the account's permanent storage area.

Requirements imprecision

- Problems arise when requirements are not precisely stated.
- Ambiguous requirements may be interpreted in different ways by developers and users.
- Consider the term '*appropriate viewers*'
  - User intention - special purpose viewer for each different document type;
  - Developer interpretation - Provide a text viewer that shows the contents of the document.

Requirements completeness and consistency

- In principle, set of requirements should be both complete and consistent.
- ***Completeness***
  - ...includes descriptions of all facilities required.
- ***Consistency***
  - ... no conflicts or contradictions.
- In practice, it is *extremely hard* to produce a complete and consistent requirements document.

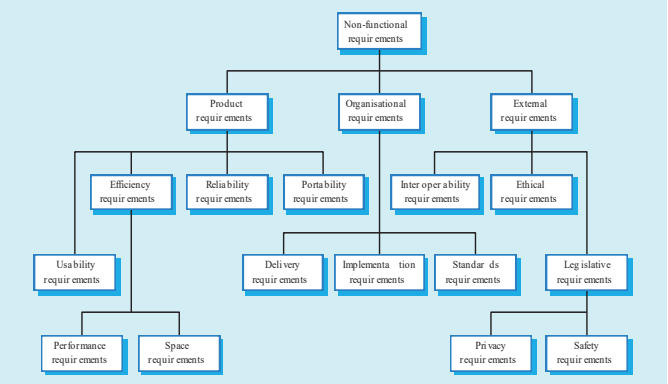
Non-functional (NF) Requirements

- ...define
  - *system properties* such as
    - reliability, response time and storage requirements, etc., and
  - *constraints* such as
    - I/O device capability, system representations, etc.
- *Process requirements* may also be specified *mandating a particular CASE system, programming language or development method*.
- ...are more critical than functional requirements. If these are not met, the system is useless.

Classifications of NF Requirements

- ***Product requirements***
  - ...specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.
- ***Organisational requirements***
  - ...are a consequence of organisational policies and procedures e.g. process standards used, implementation requirements, etc.
- ***External requirements***
  - ...arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.

Types of NF Requirements



Examples of NF Requirements

- ***Product requirement***

  8.1 The user interface for LIBSYS shall be implemented as simple *HTML without frames or Java applets*.
- ***Organisational requirement***

  9.3.2 The system development process and deliverable documents shall conform to the process and deliverables defined in *XYZCo-SP-STAN-95*.
- ***External requirement (legislative here)***

  7.6.5 The system shall not disclose any personal information about customers apart from their name and reference number to the operators of the system.

Verifiability of NF Requirements

- NF requirements ...
  - ...are usually hard to state precisely and hence to verify.
  - ...originate from *user's intention (i.e., a goal)* to state a property about the system
  - ...are *system properties* related to the entirety of the system.
- A ***verifiable*** NF requirement ...
  - ...is a statement using some measure that can be objectively tested.

Example...

- ***A system goal***
  - The system should be easy to use by experienced controllers and should be organised in such a way that user errors are minimised.
- ***A verifiable non-functional requirement***
  - Experienced controllers shall be able to use all the system functions ***after a total of two hours training***. After this training, the ***average number of errors*** made by experienced users shall ***not exceed two per day***.

NF Requirements measures

Property	Measure
Speed	Processed transactions/second User/Event response time Screen refresh time
Size	Mega Bytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Requirements interaction

- ***Conflicts between different NF requirements*** are possible in complex systems.
- Example: *Spacecraft system*
  - To minimise weight: use a minimum number of separate chips in the system.
  - To minimise power consumption: use lower-power chips meaning more chips compared with when useing higher-power chips.
  - **Conflict(!!!)**
  - **Question**: *Which is the most critical requirement?*

Domain requirements

- ...originate from the *application domain* and
- ...describe *system characteristics* that reflect the domain.
- ...may be
  - ...new functional requirements, or
  - ...constraints on existing requirements or
  - ...defining specific computations.

Library system domain requirements (Examples 1&2)

1) There shall be a standard user interface to all databases which shall be based on the *Z39.50 standard*.
2) Because of *copyright restrictions*, some documents must be deleted immediately on arrival. Depending on the user's requirements, these documents will either be printed locally on the system server for manually forwarding to the user or routed to a network printer.

Train protection system (Example 3)

3) The deceleration of the train shall be computed as:
   - $D_{train} = D_{control} + D_{gradient}$

   where $D_{gradient}$ is $9.81ms^2$ * compensated gradient/alpha and where the values of $9.81ms^2$ /alpha are known for *different types of train*.

Domain requirements problems

- *Understandability*
  - Requirements are expressed in the language of the application domain;
  - This is often not understood by software engineers developing the system.
- *Implicitness*
  - Domain specialists understand the area so well that they do not think of making the domain requirements explicit.

User requirements

- ... should describe functional and non-functional requirements in such a way that they are *understandable by system users who don't have detailed technical knowledge*.
- ... are *defined using natural language, tables and diagrams* as these can be *understood by all users*.

Problems with natural language

- *Lack of clarity*
  - Precision is difficult without making the document difficult to read.
- *Requirements confusion*
  - Functional and non-functional requirements tend to be mixed-up.
- *Requirements amalgamation*
  - Several different requirements may be expressed together.

System requirements

- More detailed specifications of system functions, services and constraints than user requirements.
- They are intended to be a basis for designing the system.
- They may be incorporated into the system contract.
- System requirements may be defined or illustrated using system models.

Requirements and design

- In principle, *requirements* should state ***what*** the system should do and the *design* should describe ***how*** it does this.
- In practice, requirements and design are inseparable
  - A system architecture may be designed to structure the requirements;
  - The system may inter-operate with other systems that generate design requirements;
  - The use of a specific design may be a domain requirement.

Problems with NL specification

- *Ambiguity*
  - The readers and writers of the requirement must interpret the same words in the same way. NL is naturally ambiguous so this is very difficult.
- *Over-flexibility*
  - The same thing may be said in a number of different ways in the specification.
- *Lack of modularisation*
  - NL structures are inadequate to structure system requirements.

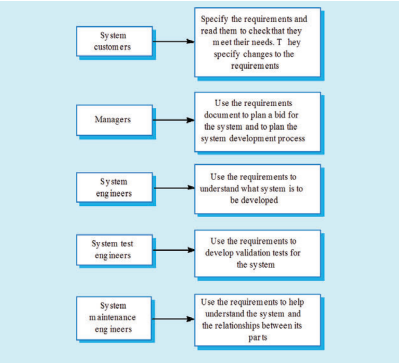
Alternatives to NL specification

- *Structured NL*
  - Form-based or tabular specifications
- *Design description languages (DDLs)*
  - ...use a language like a prog. lang. But with more abstract features to specify req.s by defining an operational model of the system.
  - Not very popular
- *Graphical Notations*
  - UML diagrams such as use-case or sequence diagrams
  - Very popular
- *Mathematical Specifications*
  - State machines or sets
  - Unambiguous, but formal and hard to understand by most customers

The requirements document

- The requirements document is the *official statement of what is required* of the system developers.
- ... includes
  - a definition of *user requirements* and
  - a specification of the *system requirements*.
- NOT a design document.
- It sets WHAT the system should do *rather than* HOW it should do it

Users of a requirements document



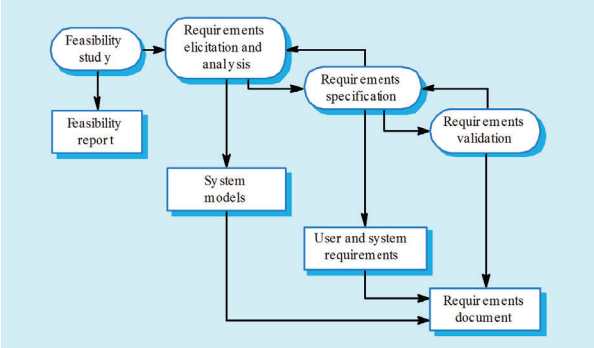
Requirements engineering

- The process of establishing the services that the customer requires from a system and the constraints under which it operates and is developed.
- The requirements themselves are the descriptions of the system services and constraints that are generated during the requirements engineering process.

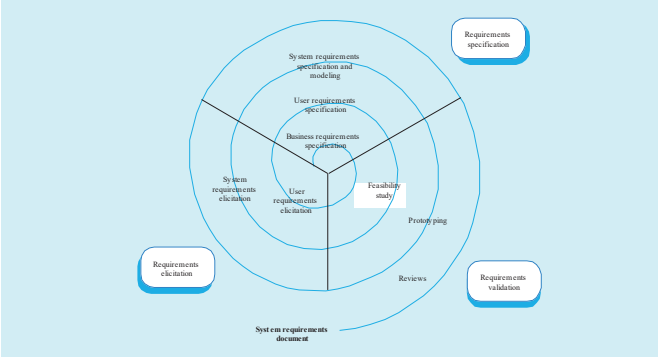
Requirements engineering processes

- The processes used for RE vary widely depending on the application domain, the people involved and the organisation developing the requirements.
- However, there are a number of generic activities common to all processes
  - Requirements elicitation;
  - Requirements analysis;
  - Requirements validation;
  - Requirements management.

The requirements engineering process



Requirements engineering



Feasibility studies

- A feasibility study decides whether or not the proposed system is worthwhile.
- A short focused study that checks
  - If the system contributes to organisational objectives;
  - If the system can be engineered using current technology and within budget;
  - If the system can be integrated with other systems that are used.

Feasibility study implementation

- Based on information assessment (what is required), information collection and report writing.
- Questions for people in the organisation
  - What if the system wasn't implemented?
  - What are current process problems?
  - How will the proposed system help?
  - What will be the integration problems?
  - Is new technology needed? What skills?
  - What facilities must be supported by the proposed system?

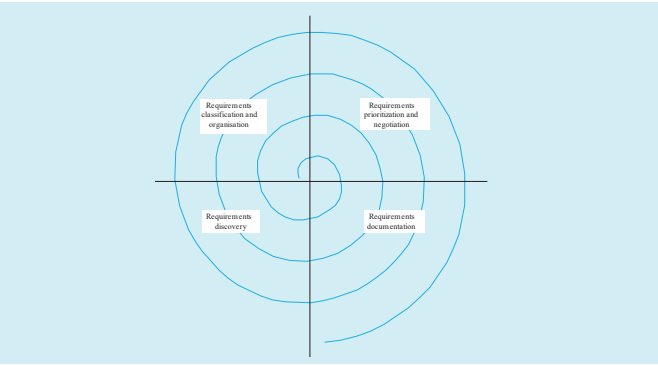
Elicitation and analysis

- A.k.a. *requirements elicitation* or *discovery*.
- *Involves technical staff working with customers to find out about the application domain, the services that the system should provide and the system's operational constraints*.
- May involve *end-users*, *managers*, *engineers involved in maintenance*, *domain experts*, *trade unions*, etc. These are called *stakeholders*.

Problems of requirements analysis

- Stakeholders
  - don't know what they really want.
  - express requirements in their own terms.
  - may have conflicting requirements.
- Organisational and political factors may influence the system requirements.
- The requirements change during the analysis process. New stakeholders may emerge and the business environment may change.

The requirements spiral



Process activities

- Requirements discovery
  - Interacting with stakeholders to discover their requirements. Domain requirements are also discovered at this stage.
- Requirements classification and organisation
  - Groups related requirements and organises them into coherent clusters.
- Prioritisation and negotiation
  - Prioritising requirements and resolving requirements conflicts.
- Requirements documentation
  - Requirements are documented and input into the next round of the spiral.

Requirements discovery

- The process of gathering information about the proposed and existing systems and distilling the user and system requirements from this information.
- Sources of information
  - documentation,
  - system stakeholders and
  - the specifications of similar systems.

ATM stakeholders

- Bank customers
- Representatives of other banks
- Bank managers
- Counter staff
- Database administrators
- Security managers
- Marketing department
- Hardware and software maintenance engineers
- Banking regulators

Viewpoints

- Viewpoints are a way of structuring the requirements to represent the *perspectives of different stakeholders*. Stakeholders may be classified under different viewpoints.
- This *multi-perspective analysis* is *important as there is no single correct way to analyse system requirements*.

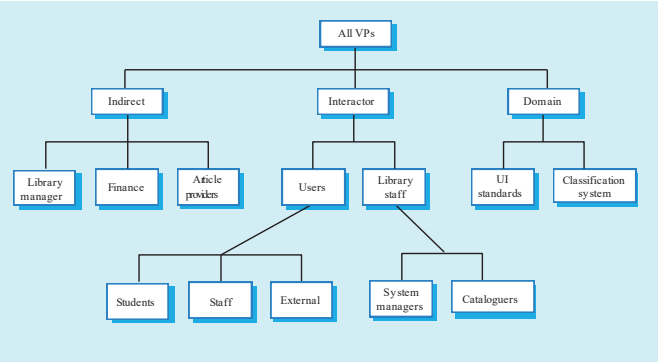
Types of viewpoint

- Interactor viewpoints
  - People or other systems that interact directly with the system. In an ATM, the customer's and the account database are interactor VPs.
- Indirect viewpoints
  - Stakeholders who do not use the system themselves but who influence the requirements. In an ATM, management and security staff are indirect viewpoints.
- Domain viewpoints
  - Domain characteristics and constraints that influence the requirements. In an ATM, an example would be standards for inter-bank communications.

Viewpoint identification

- Identify viewpoints using
  - Providers and receivers of system services;
  - Systems that interact directly with the system being specified;
  - Regulations and standards;
  - Sources of business and non-functional requirements.
  - Engineers who have to develop and maintain the system;
  - Marketing and other business viewpoints.

LIBSYS viewpoint hierarchy



Interviewing

- In formal or informal interviewing, the RE team puts questions to stakeholders about the system that they use and the system to be developed.
- There are two types of interview
  - Closed interviews where a pre-defined set of questions are answered.
  - Open interviews where there is no pre-defined agenda and a range of issues are explored with stakeholders.

Interviews in practice

- Normally a mix of closed and open-ended interviewing.
- Interviews are good for getting an overall understanding of what stakeholders do and how they might interact with the system.
- Interviews are not good for understanding domain requirements
  - Requirements engineers cannot understand specific domain terminology;
  - Some domain knowledge is so familiar that people find it hard to articulate (açıkça ifade etmek) or think that it isn't worth articulating.

Effective interviewers

- Interviewers should be open-minded, willing to listen to stakeholders and should not have pre-conceived ideas about the requirements.
- They should prompt the interviewee with a question or a proposal and should not simply expect them to respond to a question such as 'what do you want'.

Scenarios

- Scenarios are real-life examples of how a system can be used.
- They should include
  - A description of the starting situation;
  - A description of the normal flow of events;
  - A description of what can go wrong;
  - Information about other concurrent activities;
  - A description of the state when the scenario finishes.

LIBSYS scenario (1)

Initial assumption: The user has logged on to the LIBSYS system and has located the journal containing the copy of the article.

Normal: The user selects the article to be copied. He or she is then prompted by the system to either provide subscriber information for the journal or to indicate how they will pay for the article. Alternative payment methods are by credit card or by quoting an organisational account number.

The user is then asked to fill in a copyright form that maintains details of the transaction and they then submit this to the LIBSYS system.

The copyright form is checked and, if OK, the PDF version of the article is downloaded to the LIBSYS working area on the user's computer and the user is informed that it is available. The user is asked to select a printer and a copy of the article is printed. If the article has been flagged as 'print-only' it is deleted from the user's system once the user has confirmed that printing is complete.

LIBSYS scenario (2)

What can go wrong: The user may fail to fill in the copyright form correctly. In this case, the form should be re-presented to the user for correction. If the resubmitted form is still incorrect then the user's request for the article is rejected.

The payment may be rejected by the system. The user's request for the article is rejected.

The article download may fail. Retry until successful or the user terminates the session.

It may not be possible to print the article. If the article is not flagged as 'print-only' then it is held in the LIBSYS workspace. Otherwise, the article is deleted and the user's account credited with the cost of the article.

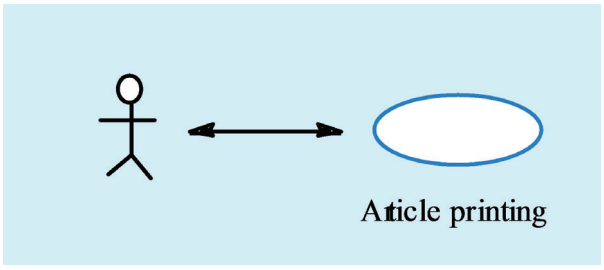
Other activities: Simultaneous downloads of other articles.

System state on completion: User is logged on. The downloaded article has been deleted from LIBSYS workspace if it has been flagged as print-only.

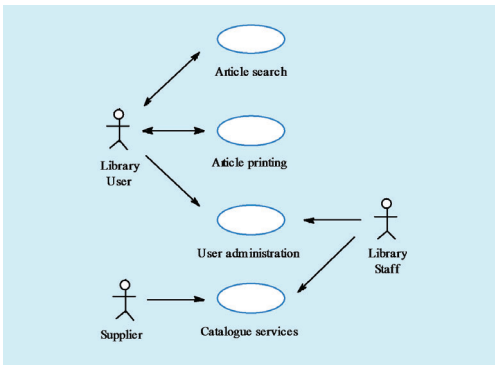
Use cases

- Use-cases are a scenario based technique in the UML which identify the actors in an interaction and which describe the interaction itself.
- A set of use cases should describe all possible interactions with the system.
- Sequence diagrams may be used to add detail to use-cases by showing the sequence of event processing in the system.

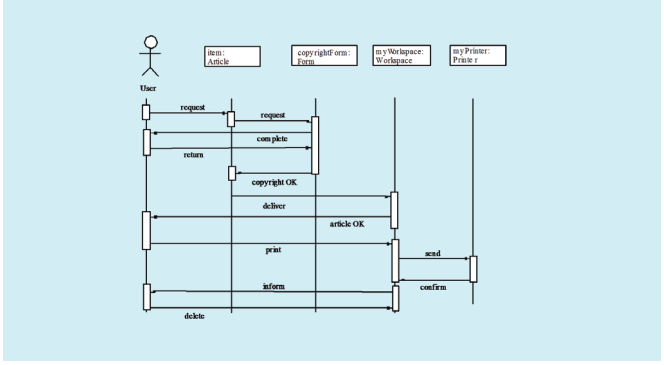
Article printing use-case



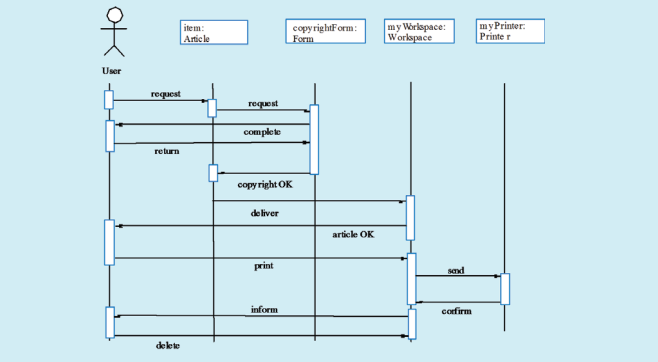
LIBSYS use cases



Article printing



Print article sequence



Social and organisational factors

- Software systems are used in a social and organisational context. This can influence or even dominate the system requirements.
- Social and organisational factors are not a single viewpoint but are influences on all viewpoints.
- Good analysts must be sensitive to these factors but currently no systematic way to tackle their analysis.

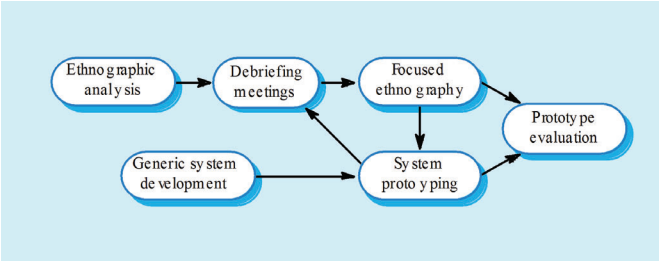
Ethnography

- A *social scientist spends a considerable time observing and analysing how people actually work*.
- *People do not have to explain or articulate their work*.
- Social and organisational factors of importance may be observed.
- Ethnographic studies have shown that work is usually richer and more complex than suggested by simple system models.

Focused ethnography

- Developed in a project studying the air traffic control process.
- Combines ethnography with prototyping.
- Prototype development results in unanswered questions focusing on ethnographic analysis.
- The problem with ethnography is that it studies existing practices which may have some historical basis which is no longer relevant.

Ethnography and prototyping



Scope of ethnography

- Requirements that are derived from the way that *people actually work rather than the way in which process definitions suggest that they should work*.
- Requirements that are derived from cooperation and awareness of other people's activities.

Requirements validation

- Concerned with demonstrating that the requirements define the system that the customer really wants.
- Requirements error costs are high so validation is very important
  - Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.

Requirements checking

- Validity. Does the system provide the functions which best support the customer's needs?
- Consistency. Are there any requirements conflicts?
- Completeness. Are all functions required by the customer included?
- Realism. Can the requirements be implemented given available budget and technology
- Verifiability. Can the requirements be checked?

Requirements validation techniques

- Requirements reviews
  - Systematic manual analysis of the requirements.
- Prototyping
  - Using an executable model of the system to check requirements. Covered in Chapter 17.
- Test-case generation
  - Developing tests for requirements to check testability.

Requirements reviews

- Regular reviews should be held while the requirements definition is being formulated.
- Both client and contractor staff should be involved in reviews.
- Reviews may be formal (with completed documents) or informal. Good communications between developers, customers and users can resolve problems at an early stage.

Review checks

- Verifiability. Is the requirement realistically testable?
- Comprehensibility. Is the requirement properly understood?
- Traceability. Is the origin of the requirement clearly stated?
- Adaptability. Can the requirement be changed without a large impact on other requirements?

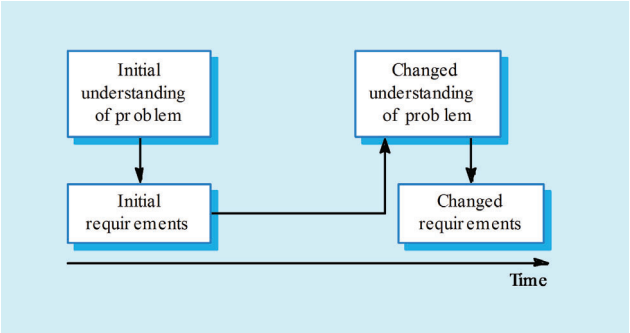
Requirements management

- Requirements management is the process of managing changing requirements during the requirements engineering process and system development.
- Requirements are inevitably incomplete and inconsistent
  - New requirements emerge during the process as business needs change and a better understanding of the system is developed;
  - Different viewpoints have different requirements and these are often contradictory.

Requirements change

- The priority of requirements from different viewpoints changes during the development process.
- System customers may specify requirements from a business perspective that conflict with end-user requirements.
- The business and technical environment of the system changes during its development.

Requirements evolution



Enduring and volatile requirements

- Enduring requirements. Stable requirements derived from the core activity of the customer organisation. E.g. a hospital will always have doctors, nurses, etc. May be derived from domain models
- Volatile requirements. Requirements which change during development or when the system is in use. In a hospital, requirements derived from health-care policy

Requirements classification

Requirement Type	Description
Mutable requirements	Requirements that change because of changes to the environment in which the organisation is operating. For example, in hospital systems, the funding of patient care may change and thus require different treatment information to be collected.
Emergent requirements	Requirements that emerge as the customer's understanding of the system develops during the system development. The design process may reveal new emergent requirements.
Consequential requirements	Requirements that result from the introduction of the computer system. Introducing the computer system may change the organisations processes and open up new ways of working which generate new system requirements
Compatibility requirements	Requirements that depend on the particular systems or business processes within an organisation. As these change, the compatibility requirements on the commissioned or delivered system may also have to evolve.

Requirements management planning

- During the requirements engineering process, you have to plan:
  - Requirements identification
    - How requirements are individually identified;
  - A change management process
    - The process followed when analysing a requirements change;
  - Traceability policies
    - The amount of information about requirements relationships that is maintained;
  - CASE tool support
    - The tool support required to help manage requirements change;

Traceability

- Traceability is concerned with the relationships between requirements, their sources and the system design
- Source traceability
  - Links from requirements to stakeholders who proposed these requirements;
- Requirements traceability
  - Links between dependent requirements;
- Design traceability
  - Links from the requirements to the design;

A traceability matrix

Req. id	1.1	1.2	1.3	2.1	2.2	2.3	3.1	3.2
1.1		D	R					
1.2			D			D		D
1.3	R			R				
2.1			R		D			D
2.2								D
2.3		R		D				
3.1								R
3.2							R	

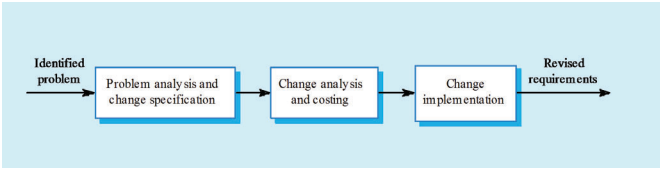
CASE tool support

- Requirements storage
  - Requirements should be managed in a secure, managed data store.
- Change management
  - The process of change management is a workflow process whose stages can be defined and information flow between these stages partially automated.
- Traceability management
  - Automated retrieval of the links between requirements.

Requirements change management

- Should apply to all proposed changes to the requirements.
- Principal stages
  - Problem analysis. Discuss requirements problem and propose change;
  - Change analysis and costing. Assess effects of change on other requirements;
  - Change implementation. Modify requirements document and other documents to reflect change.

Change management



Key points

- The requirements engineering process includes a feasibility study, requirements elicitation and analysis, requirements specification and requirements management.
- Requirements elicitation and analysis is iterative involving domain understanding, requirements collection, classification, structuring, prioritisation and validation.
- Systems have multiple stakeholders with different requirements.

Key points

- Social and organisation factors influence system requirements.
- Requirements validation is concerned with checks for validity, consistency, completeness, realism and verifiability.
- Business changes inevitably lead to changing requirements.
- Requirements management includes planning and change management.

Quiz 2 – Part 1 - Week 3

1) Who prepares the user requirements?

2) Who prepares the system requirements?

Quiz 2 – Part 2 - Week 3

1) _____ requirements are meant for a basis for a bid while _____ requirements are meant for a basis for a contract.
2) Mostly technical people are the readers of the _____ requirements.
3) _____ requirements are defined using natural languages, tables and diagrams while _____ requirements are expressed by alternatives to the natural language.

Quiz 2 – Part 3 - Week 3

1) What are the problems with natural language (NL) specification?

2) What are the alternatives to NL specification?

3) Who prepares the user requirements?

4) Who prepares the system requirements?

CSE344 Software Engineering (SWE)

Week – 5 Critical Systems & Their Specification

Critical Systems

- *Safety-critical systems*
  - Failure results in *loss of life, injury or damage to the environment*;
  - Chemical plant protection system;
- *Mission-critical systems*
  - Failure results in *failure of some goal-directed activity*;
  - Spacecraft navigation system;
- *Business-critical systems*
  - Failure results in *high economic losses*;
  - Customer accounting system in a bank;

What is the most important system property for regular SW systems?

- From the FAQ of the discussion in week 1:
- The SW process cost distribution is, roughly, as follows:
  - ***60% ↔ development costs***,
  - ***40% ↔ testing costs***.
- For *custom SW*, ***evolution*** *(or* ***maintenance****)* ***costs*** *often exceed development costs*.
-

Hence, the most essential SW system property for regular systems is SW maintainability !!!

System dependability

- For critical systems, usually the most important system property is the dependability of the system.
- The *dependability of a system* reflects the *extent of the user's confidence* that it will operate as users expect and that it will not 'fail' in normal use.
- *Usefulness (kullanışlılık)* and *trustworthiness (güvenilirlik)* are *not the same* thing. *A system does not have to be trusted to be useful.*

Importance of dependability

- *Undependable* (i.e., *unreliable, unsafe and/or insecure*) SW systems
  - may be *rejected by their users*.
  - may cause *information loss with a high recovery cost*.
- The costs of system failure may be very high.

Development methods for critical systems

- *The costs of critical system failure are so high that development methods may be used that are not cost-effective for other types of system.*
- Examples of development methods
  - ***Formal methods of software development***
  - Static analysis
  - External quality assurance

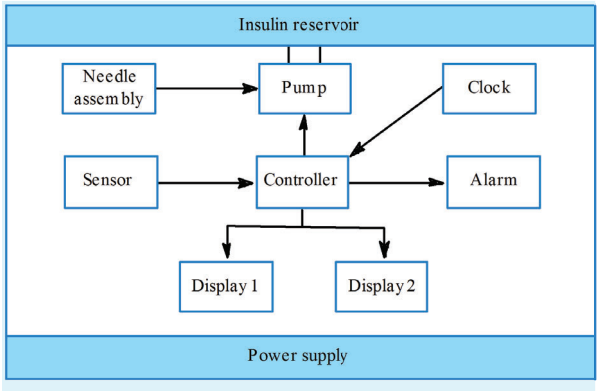
Failure types in socio-technical critical systems

- *Hardware failure*
  - Hardware fails because
    - of design and manufacturing errors or
    - components have reached the end of their natural life.
- *Software failure*
  - Software fails due to errors in its specification, design or implementation.
- *Operational failure*
  - Human operators make mistakes. Now perhaps the largest single cause of system failures.

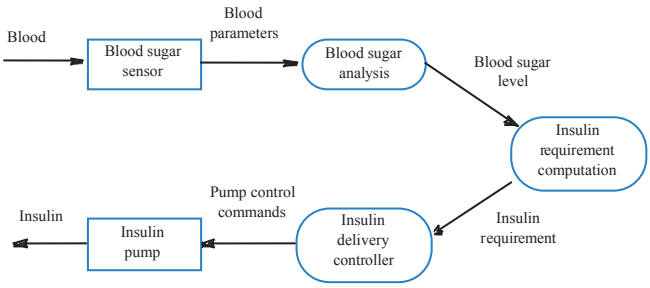
An example to critical systems: a SW-controlled insulin pump

- Used by *diabetics*
- *Simulates the function of the pancreas*
  - Manufactures *insulin*,
    - an essential hormone that *metabolises blood glucose*.
  - *Measures blood glucose* (sugar) using a micro-sensor and
  - *Computes the insulin dose required* to *metabolise* the glucose.

Insulin pump organisation



Insulin pump data-flow



Dependability requirements

- ...shall be *available* to *deliver insulin when required* to do so (*availability requirement*)
- ...shall *deliver the correct amount of insulin* to counteract (neutralize) the current level of blood sugar (*reliability requirement*).
- ...shall *never deliver excessive doses of insulin* as this is potentially *life threatening* (*safety requirement*).

Dependability

- *Dependability = Trustworthiness*.
- A *dependable* system is one *trusted by its users*.
- *Principal dimensions of dependability* are:
  - *Availability*
  - *Reliability*
  - *Safety*
  - *Security*

Principal dimensions of dependability

- *The (prob)ability of the system to ...*
  - *... deliver services **when requested** (Availability)*
  - *... deliver services **as specified** (Reliability)*
  - *... operate without catastrophic failure (Safety)*
  - *... protect itself against accidental or deliberate intrusion (Security)*

Other dependability properties

- *Repairability*
  - Reflects the *extent to which the system can be repaired in the event of a failure;*
- *Maintainability*
  - Reflects the *extent to which the system can be adapted to new requirements*;
- *Survivability*
  - Reflects the *extent to which the system can deliver services whilst under hostile attack*;
- *Error tolerance*
  - Reflects the *extent to which user input errors can be avoided and tolerated*.

Maintainability

- concerned with the ease of
  - *... repairing the system* after a failure has been discovered or
  - *... changing the system* to include new features;
- very important for critical systems as faults are often introduced into a system because of maintenance problems
- different than other dimensions of dependability because it is a *static and not a dynamic system attribute*.

Survivability

- The ability of a system *to continue to deliver its services to users under attack*
- This is an *increasingly important attribute* for distributed systems whose security can be compromised
- Survivability subsumes the notion of ***resilience*** (flexibility, *fault tolerance*) - the ability of a system to *continue in operation in spite of component failures*

Dependability vs performance

- Untrustworthy systems may be rejected by their users
- System failure costs may be very high
- It is very difficult to tune systems to make them more dependable
- It may be possible to compensate for poor performance
- Untrustworthy systems may cause loss of valuable information

Dependability costs

- Dependability costs tend to increase exponentially as increasing levels of dependability are required
- Two reasons:
  - The *use of more expensive development techniques and hardware* required to achieve higher levels of dependability
  - The *increased testing and system validation* required to convince the system client that the required levels of dependability have been achieved

Dependability economics

- *Tradeoff*: Achievement of dependability requirements costs extremely high ⇒ *more cost-effective to accept untrustworthy systems* and pay for failure costs?
- *Social and political factors*: A bad reputation for products may lead to the loss of future business.
- *System type*: for *business systems* in particular, *modest levels of dependability* OK; for *medical systems highly dependable systems* are crucial.

Availability and reliability: expressible as stochastic (probabilistic) quantity

- *Reliability*
  - The *probability of failure-free system operation over a specified time in a given environment for a given purpose as specified*
- *Availability*
  - The *probability that a system, at a point in time, will be operational and able to deliver the requested services*

Reliability terminology

Term	Description
System failure	An event that occurs at some point in time when the system does not deliver a service as expected by its users
System error	An erroneous system state that can lead to system behaviour that is unexpected by system users.
System fault	A characteristic of a software system that can lead to a system error. For example, failure to initialise a variable could lead to that variable having the wrong value when it is used.
Human error or mistake	Human behaviour that results in the introduction of faults into a system.

Faults and failures

- *Failures* are usually a result of *system errors* that are derived from *faults* in the system
- However, *faults* do not necessarily result in *system errors*
  - The faulty system state may be *transient* (temporary) and 'corrected' before an error arises
- *Errors* do not necessarily lead to system *failures*
  - The error can be *corrected* by *built-in error detection and recovery*
  - The failure can be protected against by built-in protection facilities. These may, for example, protect system resources from system errors

Perceptions of reliability

- The formal definition of reliability does not always reflect the user's perception of a system's reliability
  - The assumptions that are made about the environment where a system will be used may be incorrect
    - Usage of a system in an office environment is likely to be quite different from usage of the same system in a university environment
  - The consequences of system failures affects the perception of reliability
    - Unreliable windscreen wipers in a car may be irrelevant in a dry climate
    - Failures that have serious consequences (such as an engine breakdown in a car) are given greater weight by users than failures that are only inconvenient

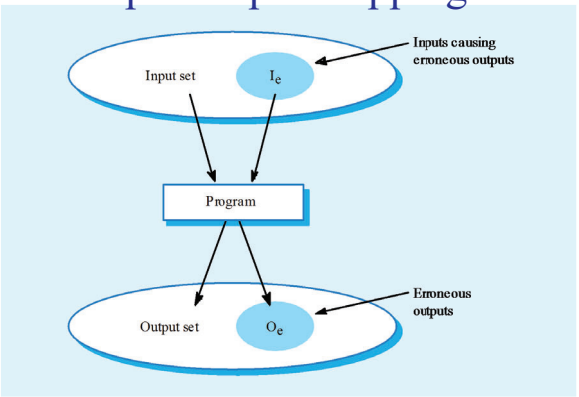
Reliability achievement

- ***Fault avoidance***
  - Development technique are used that either *minimise the possibility of mistakes* or trap mistakes before they result in the introduction of system faults
- ***Fault detection and removal***
  - Verification and validation techniques that increase the probability of *detecting and correcting errors before the system goes into service* are used
- ***Fault tolerance***
  - Run-time techniques are used to ensure that *system faults do not result in system errors and/or that system errors do not lead to system failures*

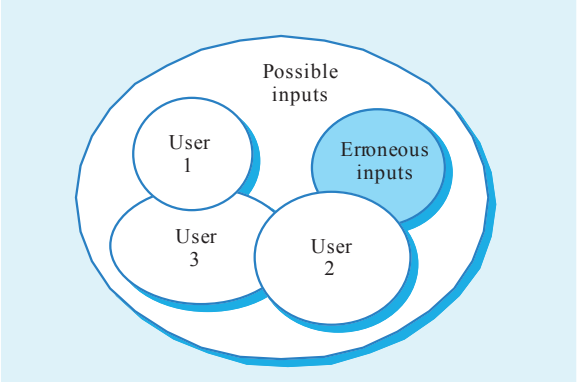
Reliability modeling

- You can model a system as an *input-output mapping* where some inputs will result in erroneous outputs
- The reliability of the system depends on the *probability that a particular input will lie in the set of inputs that cause erroneous outputs*
- Different people will use the system in different ways so this probability is not a static system attribute but depends on the system's environment

Input/output mapping



Reliability perception



Reliability improvement

- Removing X% of the faults in a system will not necessarily improve the reliability by X%. A study at IBM showed that removing 60% of product defects resulted in a 3% improvement in reliability
- Program defects may be in rarely executed sections of the code so may never be encountered by users. Removing these does not affect the perceived reliability
- A program with known faults may therefore still be seen as reliable by its users

Safety

- Safety is a property of a system that *reflects the system's ability to operate, normally or abnormally, without danger of causing human injury or death and without damage to the system's environment*
- It is *increasingly important* to consider software safety *as more and more devices incorporate software-based control systems*
- Safety requirements are *exclusive requirements* i.e. they *exclude undesirable situations* rather than specify required system services

Safety criticality

- *Primary safety-critical systems*
  - Embedded software systems whose failure can cause the associated hardware to fail and directly threaten people.
- *Secondary safety-critical systems*
  - Systems whose failure results in faults in other systems which can threaten people
- Discussion here focuses on primary safety-critical systems
  - Secondary safety-critical systems can only be considered on a one-off (every-once-in-a-while) basis

Safety and reliability

- Safety and reliability are related but distinct
  - In general, reliability and availability are necessary but not sufficient conditions for system safety
- *Reliability* is concerned with *conformance to a given specification* and delivery of service
- *Safety* is concerned with *ensuring system cannot cause damage irrespective of whether or not it conforms to its specification*

Unsafe reliable systems

- *Specification errors*
  - If the system *specification is incorrect* then the system can *behave as specified but still cause an accident*
- Hardware failures generating *spurious inputs*
  - Hard to anticipate in the specification
- Context-sensitive commands i.e. issuing the right command at the wrong time
  - Often the result of operator error

Safety terminology

Term	Definition
Accident (or mishap)	An unplanned event or sequence of events which results in human death or injury, damage to property or to the environment. A computer-controlled machine injuring its operator is an example of an accident.
Hazard	A condition with the potential for causing or contributing to an accident. A failure of the sensor that detects an obstacle in front of a machine is an example of a hazard.
Damage	A measure of the loss resulting from a mishap. Damage can range from many people killed as a result of an accident to minor injury or property damage.
Hazard severity	An assessment of the worst possible damage that could result from a particular hazard. Hazard severity can range from catastrophic where many people are killed to minor where only minor damage results.
Hazard probability	The probability of the events occurring which create a hazard. Probability values tend to be arbitrary but range from *probable* (say 1/100 chance of a hazard occurring) to implausible (no conceivable situations are likely where the hazard could occur).
Risk	This is a measure of the probability that the system will cause an accident. The risk is assessed by considering the hazard probability, the hazard severity and the probability that a hazard will result in an accident.

Safety achievement

- *Hazard avoidance*
  - The system is designed so that some classes of hazard simply cannot arise.
- *Hazard detection and removal*
  - The system is designed so that hazards are detected and removed before they result in an accident
- *Damage limitation*
  - The system includes protection features that minimise the damage that may result from an accident

Normal accidents

- Accidents in complex systems rarely caused by single cause as these systems are *resilient* (i.e., durable, resistant) to a single point of failure
  - Designing systems so that *a single point of failure does not cause an accident* is a fundamental principle of safe systems design
- Almost all accidents are a result of *combinations of malfunctions*
- Anticipating all problem combinations, especially, in software controlled systems is next to impossible so *achieving complete safety is impossible*

Quiz 2 – Part 2 - Week 4

Send answers to **cse3044marmara@gmail.com**

(20pts + 10pts + 10pts)

3. *In what different levels may the relability be achieved?*
4. *Reliability may be achieved in only one of these levels. Reliability achievement may not occur at more than one of these levels. (True/False)*
5. *Removing 3% of the faults in a system may improve the reliability by 60%. (True/False)*

Security

- ... a system property that *reflects the system's ability to protect itself from accidental or deliberate external attack*
- Security is becoming *increasingly important* as systems are *networked* so that *external access to the system* through the Internet is *possible*
- Security is an essential pre-requisite for availability, reliability and safety

Fundamental security

- *If a system* is a networked system and *is insecure then statements about its reliability and its safety are unreliable*.
- These statements depend on the executing system and the developed system being the same. However, intrusion can change the executing system and/or its data.
- *Therefore, the reliability and safety assurance is no longer valid.*

Security terminology

Term	Definition
Exposure	Possible loss or harm in a computing system. This can be loss or damage to data or can be a loss of time and effort if recovery is necessary after a security breach.
Vulnerability	A weakness in a computer-based system that may be exploited to cause loss or harm.
Attack	An exploitation of a system vulnerability. Generally, this is from outside the system and is a deliberate attempt to cause some damage.
Threats	Circumstances that have potential to cause loss or harm. You can think of these as a system vulnerability that is subjected to an attack.
Control	A protective measure that reduces a system vulnerability. Encryption would be an example of a control that reduced a vulnerability of a weak access control system.

Damage from insecurity

- *Denial of service*
  - The system is forced into a state where normal services are unavailable or where service provision is significantly degraded
- *Corruption of programs or data*
  - The programs or data in the system may be modified in an unauthorised way
- *Disclosure of confidential information*
  - Information that is managed by the system may be exposed to people who are not authorised to read or use that information

Security assurance

- *Vulnerability avoidance*
  - The system is designed so that vulnerabilities do not occur. For example, if there is no external network connection then external attack is impossible
- *Attack detection and elimination*
  - The system is designed so that attacks on vulnerabilities are detected and neutralised before they result in an exposure. For example, virus checkers find and remove viruses before they infect a system
- *Exposure limitation*
  - The system is designed so that the adverse consequences of a successful attack are minimised. For example, a backup policy allows damaged information to be restored

Dependability requirements

- *Functional requirements* to define error checking and recovery facilities and protection against system failures.
- *Non-functional requirements* defining the required reliability and availability of the system.
- *Excluding requirements* that define states and conditions that must not arise.

Quiz 2 – Part 3 - Week 4 (by 10:31)

Send answers to **cse3044marmara@gmail.com**
(25pts + 15pts)

6. *How can an unsafe system be reliable?*

7. *Which SW process model is the most appropriate model for critical SW development?*

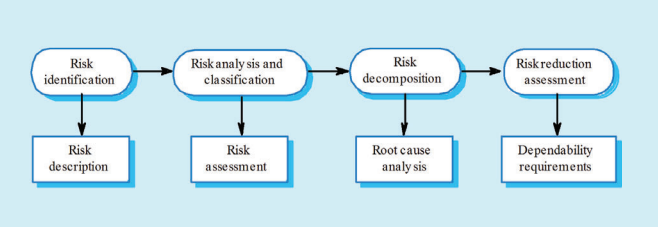
Critical systems specification...

- ...***risk-driven***.
- ...widely used in safety and security-critical systems.
- Aim of specification: to *understand the risks* (safety, security, etc.) faced by the system and to *define requirements reducing these risks*.

Stages of risk-based analysis

- *Risk identification*
  - Identify potential risks that may arise.
- *Risk analysis and classification*
  - Assess the seriousness of each risk.
- *Risk decomposition*
  - Decompose risks to discover their potential *root causes*.
- *Risk reduction assessment*
  - Define how each risk must be eliminated or reduced when the system is designed.

Risk-driven specification



Risk identification

- Risks for
  - Safety-critical systems: *hazards that can lead to accidents*.
  - Security-critical systems: *potential attacks on the system*.
- First: *identify risk classes*, and
- Second: *position risks in these classes*
  - Service failure;
  - Electrical risks;
  - …

Insulin pump risks

- Insulin overdose (service failure).
- Insulin underdose (service failure).
- Power failure due to exhausted battery (electrical).
- Electrical interference with other medical equipment (electrical).
- Poor sensor and actuator contact (physical).
- Parts of machine break off in body (physical).
- Infection caused by introduction of machine (biological).
- Allergic reaction to materials or insulin (biological).

Risk analysis and classification

- ...concerned with understanding the likelihood of possible risks and the potential consequences in case of any accident or incident.
- Risks categories:
  - ***Intolerable***. Must *never arise* or result in an accident
  - ***As low as reasonably practical(ALARP).*** Must *minimise the possibility of risk* given cost and schedule constraints
  - ***Acceptable.*** The *consequences of the risk are acceptable* and no extra costs should be incurred to reduce hazard probability

Risk assessment

- Estimate
  - risk probability and
  - risk severity.
- Normally not possible to do this precisely so relative values are used such as ‘unlikely’, ‘rare’, ‘very high’, etc.
- The aim must be to exclude risks that are likely to arise or that have high severity.

Risk assessment - insulin pump

Identified hazard	Hazard probability	Hazard severity	Estimated risk	Acceptability
1. Insulin overdose	Medium	High	High	Intolerable
2. Insulin underdose	Medium	Low	Low	Acceptable
3. Power failure	High	Low	Low	Acceptable
4. Machine incorrectly fitted	High	High	High	Intolerable
5. Machine breaks in patient	Low	High	Medium	ALARP
6. Machine causes infection	Medium	Medium	Medium	ALARP
7. Electrical interference	Low	High	Medium	ALARP
8. Allergic reaction	Low	Low	Low	Acceptable

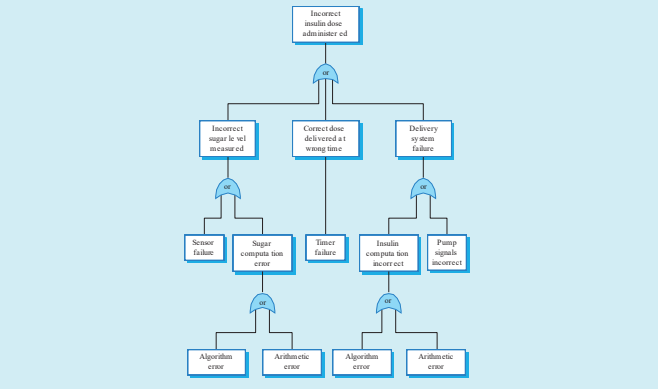
Risk decomposition...

- ...concerned with discovering the *root causes* of risks in a particular system.
- Two techniques...
  - *inductive, bottom-up techniques*.
    - start with a proposed *system failure*, and
    - assess the hazards that could arise from that failure;
  - *deductive, top-down techniques.*
    - start with a hazard, and
    - deduce what the causes of this could be.

Fault-tree analysis

- A *deductive top-down* technique.
- Put the risk or *hazard at the root of the tree* and *identify the system states that could lead to that hazard*.
- Where appropriate, *link these with 'and' or 'or'* conditions.
- Goal: *minimise the number of single causes of system failure*.

Insulin pump fault tree



Risk reduction assessment

- Aim:
  - to identify dependability requirements that
    - specify how the risks should be managed, and
    - ensure that accidents/incidents do not arise.
- Risk reduction strategies
  - *Risk avoidance*;
  - *Risk detection and removal*;
  - *Damage limitation*.

Strategy use

- Normally, in critical systems, a *mix of risk reduction strategies* are used.
- In a chemical plant control system, the system will include sensors to detect and correct excess pressure in the reactor (*Risk detection and removal*).
- However, it will also include an independent protection system that opens a relief valve if dangerously high pressure is detected (*Damage limitation*).

Insulin pump - software risks

- Arithmetic error
  - A computation causes the value of a variable to overflow or underflow;
  - Maybe include an exception handler for each type of arithmetic error.
- Algorithmic error
  - Compare dose to be delivered with previous dose or safe maximum doses. Reduce dose if too high.

Safety requirements - insulin pump

SR1: *The system shall not deliver a single dose of insulin that is greater than a specified maximum dose for a system user.*
SR2: *The system shall not deliver a daily cumulative dose of insulin that is greater than a specified maximum for a system user.*
SR3: *The system shall include a hardware diagnostic facility that shall be executed at least 4 times per hour.*
SR4: *The system shall include an exception handler for all of the exceptions that are identified in Table 3 (an imaginary table stating exceptions in Req. Spec. Doc.).*
SR5: *The audible alarm shall be sounded when any hardware or software anomaly is discovered and a diagnostic message as defined in Table 4 should be displayed.*
SR6: *In the event of an alarm in the system, insulin delivery shall be suspended until the user has reset the system and cleared the alarm.*

Safety specification

- The safety requirements of a system should be ***separately specified***.
- These requirements should be ***based on an analysis of the possible hazards and risks*** as previously discussed.
- Safety requirements usually ***apply to the system as a whole*** rather than to individual sub-systems. In systems engineering terms, the ***safety*** of a system ***is an emergent property***.

Safety requirements

- *Functional safety requirements*
  - … define the safety functions of the protection system i.e. ***define how the system should provide protection***.
- *Safety integrity requirements*
  - … define the reliability and availability of the protection system. They are based on expected use and are classified using a safety integrity level from 1 to 4.

Security specification

- *Similarities to safety specification*
  - Not possible to specify security requirements quantitatively;
  - The requirements are often 'shall not' rather than 'shall' requirements. (similarity in being ***excluding*** requirements.)
- *Differences*
  - No well-defined notion of a security life cycle for security management; *No standards*;
  - *Generic threats* rather than system specific hazards;
  - *Mature security technology (encryption, etc.)*. However, there are *problems in transferring this into general use*;
  - The *dominance of a single supplier* (Microsoft) means that huge numbers of systems may be affected by security failure.

Stages in security specification

- Asset identification and evaluation
  - The assets (data and programs) and their required degree of protection are identified. The degree of required protection depends on the asset value: e.g., a password file (say) is more valuable than a set of public web pages.
- Threat analysis and risk assessment
  - Possible security threats are identified and the risks associated with each of these threats is estimated.
- Threat assignment
  - Identified threats are related to the assets so that, for each identified asset, there is a list of associated threats.

Stages in security specification

- Technology analysis
  - Available security technologies and their applicability against the identified threats are assessed.
- Security requirements specification
  - The security requirements are specified. Where appropriate, these will explicitly identify the security technologies that may be used to protect against different threats to the system.

LIBSYS security requirements

SEC1: All system users shall be identified using their library card number and personal password.
SEC2: Users privileges shall be assigned according to the class of user (student, staff, library staff).
SEC3: Before execution of any command, LIBSYS shall check that the user has sufficient privileges to access and execute that command.
SEC4: When a user orders a document, the order request shall be logged. The log data maintained shall include the time of order, the user's identification and the articles ordered.
SEC5: All system data shall be backed up once per day and backups stored off-site in a secure storage area.
SEC6: Users shall not be permitted to have more than 1 simultaneous login to LIBSYS.

System reliability specification

- *Hardware reliability*
  - What is the probability of a hardware component failing and how long does it take to repair that component?
- *Software reliability*
  - How likely is it that a software component will produce an incorrect output? Software failures are different from hardware failures in that software does not wear out. It can continue in operation even after an incorrect result has been produced.
- *Operator reliability*
  - How likely is it that the operator of a system will make an error?

Functional reliability requirements

- A predefined range for all values that are input by the operator shall be defined and the system shall check that all operator inputs fall within this predefined range.
- The system shall check all disks for bad blocks when it is initialised.
- The system must use *N-version programming (Fault tolerant SW)* to implement the braking control system.
- The system must be implemented in a safe subset of Ada and checked using static analysis.

Non-functional reliability specification

- The level of system reliability required *should be verifiable*, hence, *expressed quantitatively*.
- Reliability is a *dynamic system attribute*- reliability specifications *related to the source code* are *meaningless such as the example below:*
  - *No more than N faults/1000 lines*;
  - This is only useful for a post-delivery process analysis where you are trying to assess how good your development techniques are.
- An appropriate reliability metric should be chosen to specify the overall system reliability.

Reliability metrics

- ***Probability of failure on demand (POFOD):*** *The likelihood that the system will fail when a service request is made.* A POFOD of 0.001 means that 1 out of a thousand service requests may result in failure.
- ***Rate of failure occurrence (ROCOF):*** *The frequency of occurrence with which unexpected behaviour is likely to occur.* A ROCOF of 2/100 means that 2 failures are likely to occur in each 100 operational time units (tu). This metric is sometimes called the *failure intensity*.
- ***Mean time to failure (MTTF):*** *The average time between two consecutive system failures.* An MTTF of 500 means that 1 failure can be expected every 500 tus.
- ***Availability (AVAIL):*** *The probability that the system is available for use at a given time.* Availability of 0.998 means that in every 1000 tus, the system is likely to be available for 998 tus.

Steps to a reliability specification

1. For each sub-system: *analyse consequences of possible system failures*.
2. From *system failure analysis*: *partition failures into appropriate classes (regarding the seriousness of their consequence)*.
3. For each *failure class* identified: *set out the reliability using an appropriate metric*.
   - *Possible to use different metrics for different reliability requirements*.
4. To reduce the chances of critical failures: *identify functional reliability requirements*

Bank auto-teller system

- Each machine in a network is used 300 times a day
- Bank has 1000 machines
- Lifetime of software release is 2 years
- Each machine handles about 200, 000 transactions
- About 300, 000 database transactions in total per day

Reliability specification for an ATM

Failure class	Example	Reliability metric
Permanent, non-corrupting.	The system fails to operate with any card that is input. Software must be restarted to correct failure.	ROCOF 1 occurrence/1000 days
Transient, non-corrupting	The magnetic stripe data cannot be read on an undamaged card that is input.	ROCOF 1 in 1000 transactions
Transient, corrupting	A pattern of transactions across the network causes database corruption.	Unquantifiable! Should never happen in the lifetime of the system

Specification validation

- It is impossible to empirically validate very high reliability specifications.
- No database corruptions means POFOD of less than 1 in 200 million service requests.
- If a transaction takes 1 second, then simulating one day's transactions takes 3.5 days.
- It would take longer than the system's lifetime to test it for reliability.

Quiz 1 – Week 5

Send answers to **cse3044marmara@gmail.com**

(20pts + 10pts + 20pts)

1. Why is a *reliability requirement* such as «No more than N faults/1000 lines» is meaningless?
2. List at least three reliability metrics!
3. What type of requirements are safety and security requirements in general?

SOFTWARE DESIGN (YAZILIM TASARIMI)

Week 5

Principles of Design

- *Design emerges from a need to solve a problem.*
- *Design is a creative process to meet the need to solve the problem.*
- *Design has two phases:*
  - *Inspiration phase*
  - *Expression or communication phase.*
- *Design – as applied by nature – is a bottom-up process.*

Design: Inspiration Phase

- *The original (i.e., inspired) idea or thought is disordered or chaotic.*
- *It is not expressible/communicatible as inspired.*
- *A powerful communication basis is required to express the inspired thought(s) in an organized manner.*

Design: Communication Phase

- *Idea or thought is disorganized as inspired.*
- *Words form a tool to communicate inspired thoughts.*
- *Words are components to express/communicate thoughts in a comprehensible manner.*
- *Designer is not confined in her inspirations; but restricted to the expessive power of the words she selects to communicate her thought(s).*
- *The quality of the communicated thought depends upon the expressive power of the words and the potential of the designer to use words.*

Conclusion from Last Page

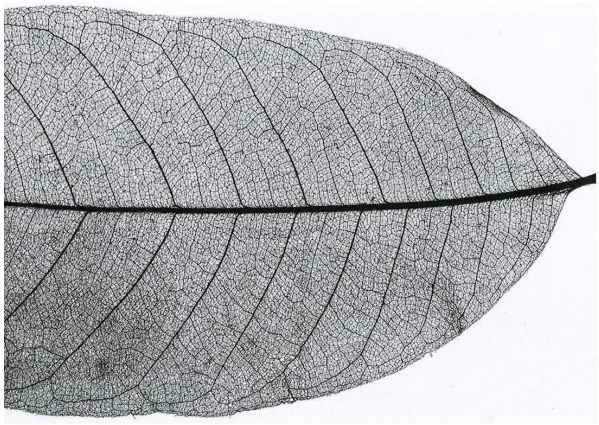
It is inevitable that the designer should have a well-established basis of knowledge on the set of communication tool(s) to use them to her best.

Nature's Tradition of Perfect Design

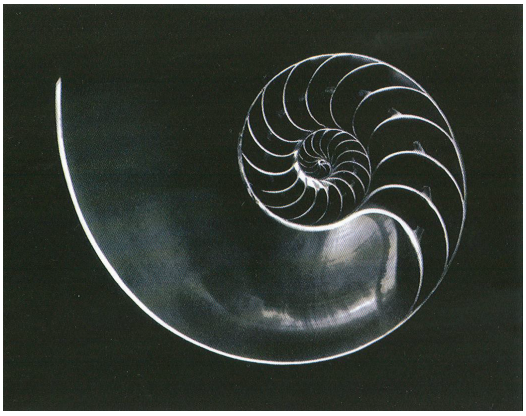
- *Nature designs perfectly since nature bases its designs upon the (to a significant extent unknown) internal reasons originating from an incredible level of balance in an enourmously diverse system of adaptable (evolving) organisms.*

Nature's Multi-Levelled Approach to Design

- *Nature has created the tiger to do whatever it achieves to survive in its environment; through **evolution** nature has excelled its design (i.e., all aspects ranging from its most detailed cellular to most general systemic level) to progressively fit its environment.*
- *The same is valid for a leaf or a snail or us.*



Photography: Andreas Feininger



Photography: Andreas Feininger

Nature Designs in a Bottom-up Fashion!

- *Nature's way of design ranging from the most detailed cellular to most general systemic level to provide an organism with all necessary and sufficient properties to survive its current environment is explained by its **bottom-up** approach to design along with its equipping these organisms with evolvability.*

Means of SW Design

- *UML*
- *Prototyping language (if required)*
- *Formal Language, corresponding IDE*
- *Environment: O/S, hardware, network.*
- *Graphic tools (if necessary)*
- *Others?*

SW Design: A Top-Down & Bottom-up Process

- *Designer is required to establish a sufficient background on all tools employed in any part of SW design.*
- *Establishing the required level of background on tools of use, the designer naturally follows a bottom-up approach to SW design.*
- *Time that nature has is what designer does not. Hence, top-down approach is an option as a structuring approach to design in scarcity of time*

Key Points

- *Look for solutions as simple as possible;*
- *Nature's choice is also for as simple/optimal as can be. We appreciate this fact from our everyday experience on living organisms that they have the simplest necessary equipment for the given functionality;*
- *Remember! Among a number of solution alternatives to a problem, the best one is the simplest one! (Occam's razor!);*

Key Points

- *Learn all necessary means of SW design (i.e., those we have considered on page 11) to your best and expand your experience on using them;*
- *Just as one who possesses a large vocabulary and is talented (and/or educated) to use words meaningfully and correctly can more powerfully express her/his thoughts than an ordinary person, a SW engineer well equipped with/experienced on necessary tools is more likely to come up with better SW designs in general.*

Architectural Design

Architectural Design

- ... is the design stage for identifying
  - the *sub-systems* making up a system and
  - the framework for sub-system *control and communication*.

Architectural design ...2

- an *early stage* of the *system design*.
- a *link between specification and design*.
- ... involves *identifying major system components and their communications*.
- especially *cost-effective for large scale systems*.

Advantages of explicit architecture

- ***Stakeholder communication***
  - a *focus of discussion* by system stakeholders.
- ***System analysis***
  - Architectural design at an early stage requires system analysis. This in turn makes analysis possible of *whether the system can meet its non-functional requirements*.
- ***Large-scale reuse***
  - architecture reusable?

Architecture and system characteristics

- ***Performance***
  - Localise critical operations and minimise communications. Use larger rather than finer components.
- ***Security***
  - Use layered architecture with critical assets in the inner layers.
- ***Safety***
  - Localise safety-critical features in a small number of sub-systems.
- ***Availability***
  - Include redundant components and mechanisms for fault tolerance.
- ***Maintainability***
  - Use finer, replaceable components.

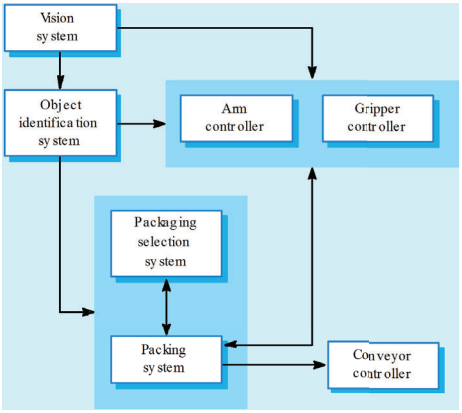
Architectural conflicts

- A ***high performance and maintainable*** system?
- ***Availability*** (i.e., redundant data) and ***security*** (i.e., layered architecture) together?
- A ***safe*** (safety-related features means more communication) and ***high performance*** system?

System structuring

- decomposing the system into *interacting sub-systems*.
- normally expressed as a *block diagram* presenting an overview of the system structure.
- showing how sub-systems
  - *share data*,
  - *are distributed* and
  - *interface with each other*.

Packing robot control system



Architectural design decisions

- Is there a generic application architecture that can be used?
- How will the system be distributed?
- What architectural styles are appropriate?
- What approach will be used to structure the system?
- How will the system be decomposed into modules?
- What control strategy should be used?
- How will the architectural design be evaluated?
- How should the architecture be documented?

Architectural models

- ***Static structural model*** that shows the major system components.
- ***Dynamic process model*** that shows the process structure of the system.
- ***Interface model*** that defines sub-system interfaces.
- ***Relationships model*** such as a data-flow model that shows sub-system relationships.
- ***Distribution model*** that shows how sub-systems are distributed across computers.

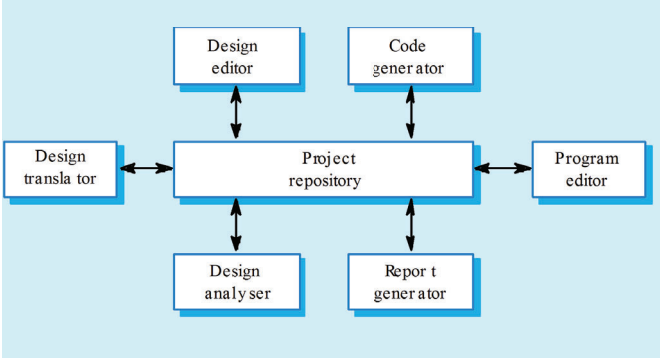
System organisation

- Reflects the basic strategy that is used to structure a system into its subsystems.
- Three organisational styles are widely used:
  - A ***shared data repository*** style;
  - A ***shared services and servers*** style;
  - An ***abstract machine or layered*** style.

The repository model

- Sub-systems exchange data in two ways:
  - ***Shared data is held in a central database or repository*** and may be accessed by all sub-systems;
  - ***Each sub-system maintains its own database and passes data explicitly to other sub-systems***.
- When ***large amounts of data*** are to be shared, the ***repository model*** of sharing is most commonly used.

CASE toolset architecture



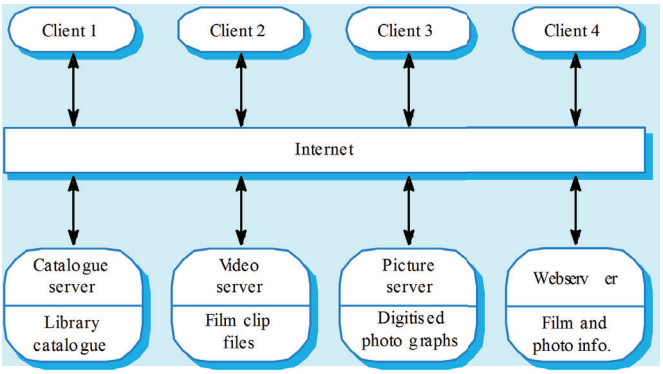
Repository model characteristics

- ***Advantages***
  - Efficient way to share large amounts of data;
  - Sub-systems need not be concerned with how data is produced. Centralised management e.g. backup, security, etc.
- ***Disadvantages***
  - Sub-systems must agree on a repository data model. Inevitably a compromise;
  - Data evolution is difficult and expensive;
  - Difficult to distribute efficiently.

Client-server model

- Distributed system model which shows *how data and processing is distributed across a range of components*.
- *Set of stand-alone servers* which provide specific services such as printing, data management, etc.
- *Set of clients* which request for these services.
- Network which allows clients to access servers.

Film and picture library



Client-server characteristics

- ***Advantages***
  - Distribution of data is straightforward;
  - Makes effective use of networked systems. May require cheaper hardware;
  - Easy to add new servers or upgrade existing servers.
- ***Disadvantages***
  - No shared data model so sub-systems use different data organisation. Data interchange may be inefficient;
  - Redundant management in each server;
  - No central register of names and services - it may be hard to find out what servers and services are available.

Abstract machine (layered) model

- Used to *model the interfacing of sub-systems*.
- Organises the *system into a set of layers* (or abstract machines) each of which provide *a set of services*.
- Supports the *incremental development* of sub-systems in different layers. When a layer interface changes, only the adjacent layer is affected.
- However, often artificial to structure systems in this way.

Modular decomposition styles

- Styles of *decomposing sub-systems into modules*.
- No rigid distinction between system organisation and modular decomposition.

Sub-systems and modules

- A *sub-system* is a system in its own right whose operation is ***independent*** of the services provided by other sub-systems.
- A *module* is a system component that provides services to other components but would ***not*** normally be considered as a ***separate*** system.

Modular decomposition

- Two modular decomposition models covered
  - An ***object model*** where the system is decomposed into interacting object;
  - A pipeline or data-flow model where the system is decomposed into ***functional modules*** which transform inputs to outputs.
- If possible, decisions about concurrency should be delayed until modules are implemented.

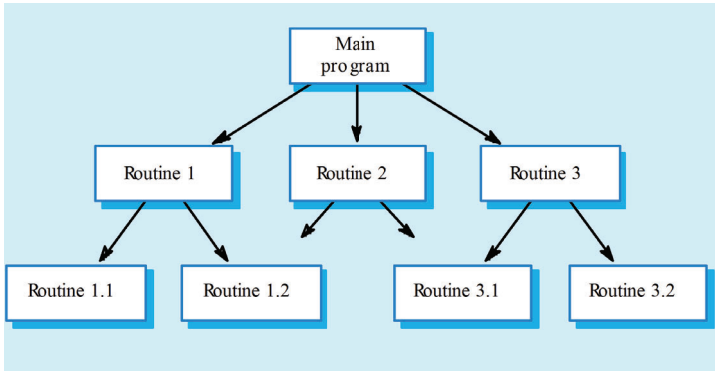
Control styles

- Ways of the control flow between sub-systems. Two ways:
  - ***Centralised control***
  - ***Event-based control***

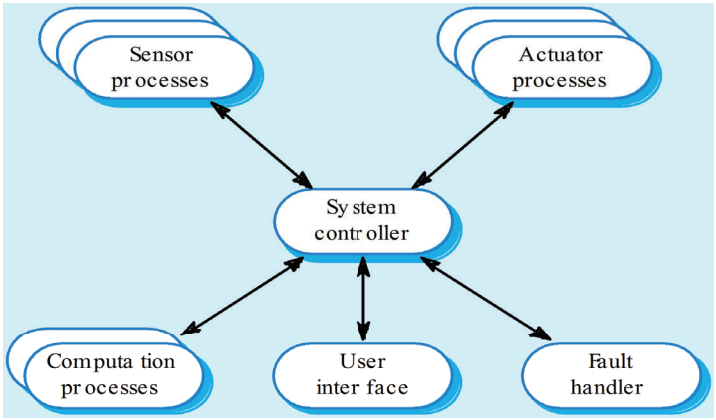
Centralised control

- A control sub-system takes responsibility for managing the execution of other sub-systems.
- ***Call-return model***
  - Top-down subroutine model
  - Control starts at the root of a subroutine tree and moves downwards.
  - Applicable to sequential systems.
- ***Manager model***
  - Applicable to concurrent systems.
  - One system component coordinates other system processes.

Call-return model



Real-time system control



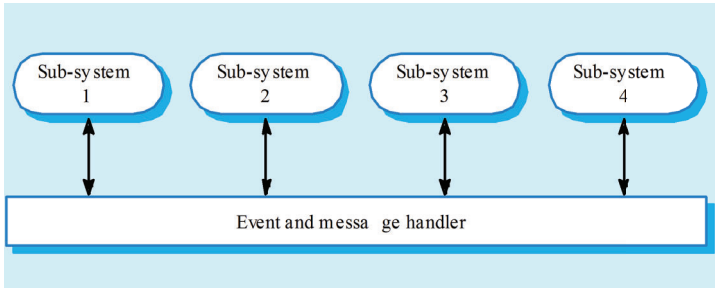
Event-driven systems

- The control of the sub-systems processing the event is driven by externally generated events
- Two primary event-driven models
  - ***Broadcast models***. An *event is broadcast to all sub-systems*. Any sub-system which can handle the event may do so;
  - ***Interrupt-driven models***. *Used in real-time systems* where interrupts are detected by an interrupt handler and passed to some other component for processing.
- Other event driven models include spreadsheets and production systems.

Broadcast model

- Effective in *integrating sub-systems on different computers* in a network.
- *Sub-systems register an interest in specific events*. When these occur, control is transferred to the sub-system which can handle the event.
- Control policy is not embedded in the event and message handler. Sub-systems decide on events of interest to them.
- However, sub-systems don't know if or when an event will be handled.

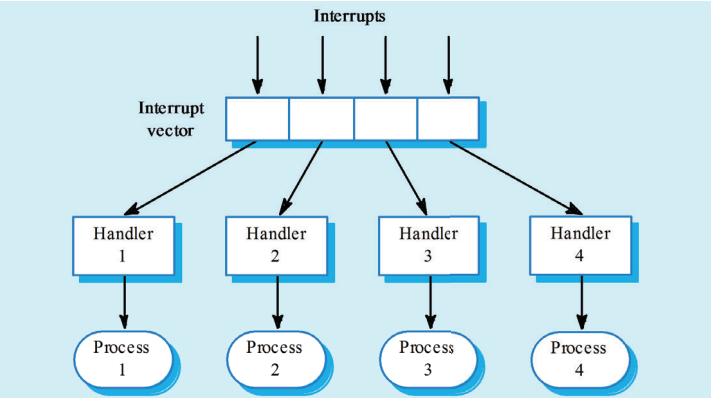
Selective broadcasting



Interrupt-driven systems

- *Used in real-time systems* where *fast response to an event is essential*.
- There are known *interrupt types with a handler* defined for each type.
- Each type is associated with *a memory location and a hardware switch causes transfer to its handler*.
- Allows *fast response but complex to program and difficult to validate*.

Interrupt-driven control



Distributed System Design

Distributed systems

- Virtually all large computer-based systems are now distributed systems.
- Information processing is *distributed over several computers* rather than confined to a single machine.
- *Distributed software engineering* is therefore very important for enterprise computing systems.

Distributed system characteristics

- ***Resource sharing***
  - Sharing of hardware and software resources.
- ***Openness***
  - Use of equipment and software from different vendors.
- ***Concurrency***
  - Concurrent processing to enhance performance.
- ***Scalability***
  - Increased throughput by adding new resources.
- ***Fault tolerance***
  - The ability to continue in operation after a fault has occurred.

Distributed system disadvantages

- ***Complexity***
  - Typically, ***distributed systems are more complex*** than centralised systems.
- ***Security***
  - ***More susceptible*** to external attack.
- ***Manageability***
  - ***More effort required*** for system management.
- ***Unpredictability***
  - Unpredictable responses depending on the system organisation and ***network load***.

Distributed systems architectures

- ***Client-server architectures***
  - *Distributed services*
    - provided by *servers*
    - requested and used by *clients*.
- ***Distributed object architectures***
  - *No distinction between clients and servers.*

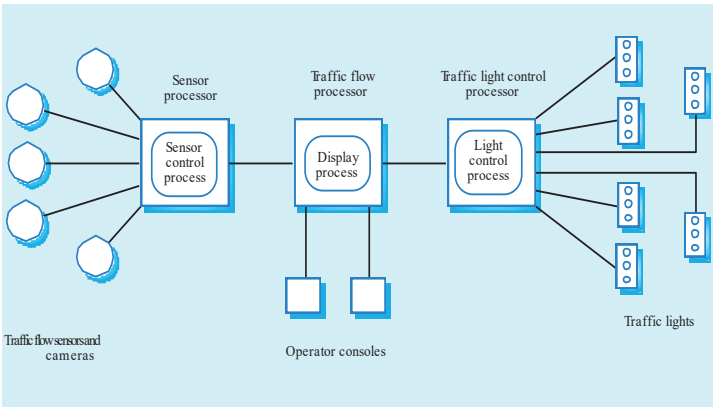
Middleware

- *Software that manages and supports the different components of a distributed system*. In essence, it sits in the ***middle*** of the system.
- Middleware is usually ***off-the-shelf*** rather than specially written software.
- Examples
  - Transaction processing monitors;
  - Data converters;
  - Communication controllers.

Multiprocessor architectures

- *Simplest distributed system* model.
- System composed of *multiple processes* which may (but need not) *execute on different processors*.
- Architectural model of many *large real-time systems*.
- *Process to processor distribution*
  - *either pre-ordered*
  - *or under the control of a dispatcher.*

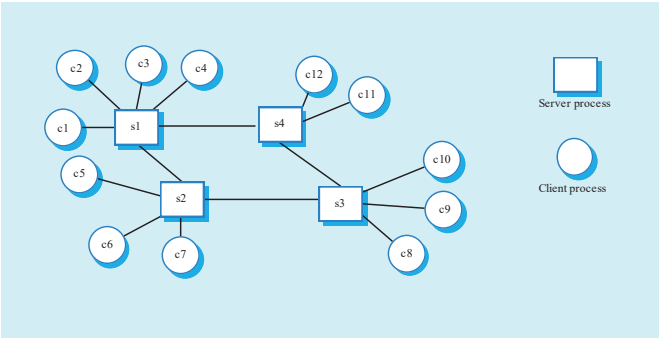
A multiprocessor traffic control system



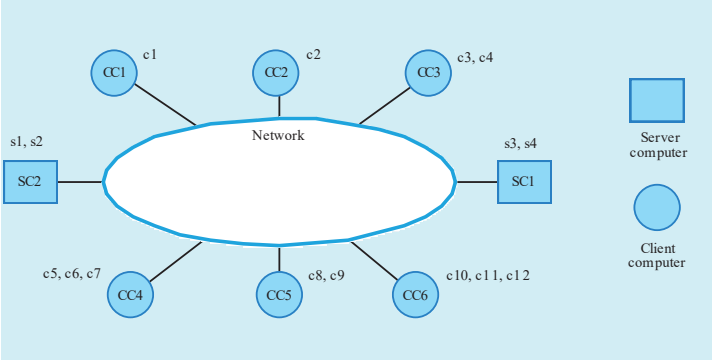
Client-server architectures

- The application is modelled as a set of services that are provided by servers and a set of clients that use these services.
- Clients know of servers but servers need not know of clients.
- Clients and servers are logical processes
- The mapping of processors to processes is not necessarily 1 : 1.

A client-server system



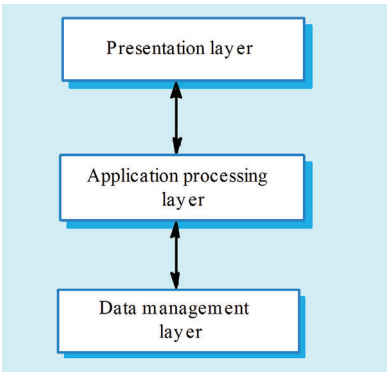
Computers in a C/S network



Layered application architecture

- ***Presentation layer***
  - *presenting the results of a computation* to users and
  - *collecting user inputs*.
- ***Application processing layer***
  - *providing application specific functionality* e.g., in a banking system, banking functions such as open account, close account, etc.
- ***Data management layer***
  - *managing the system databases*.

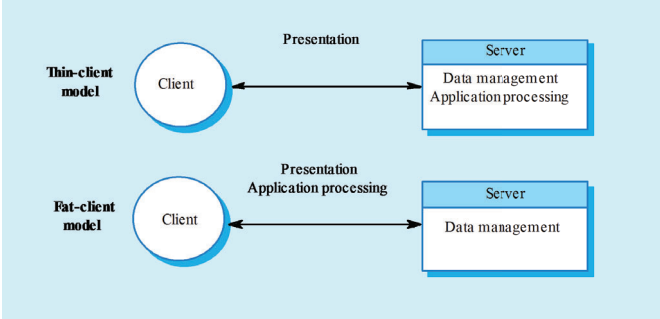
Application layers



Thin and fat clients

- ***Thin-client model***
  - *Application processing and data management is on the server.*
  - *Client simply responsible for running presentation software.*
- ***Fat-client model***
  - *Server only responsible for data management.*
  - *Client realizes application logic and interactions with the system user.*

Thin and fat clients



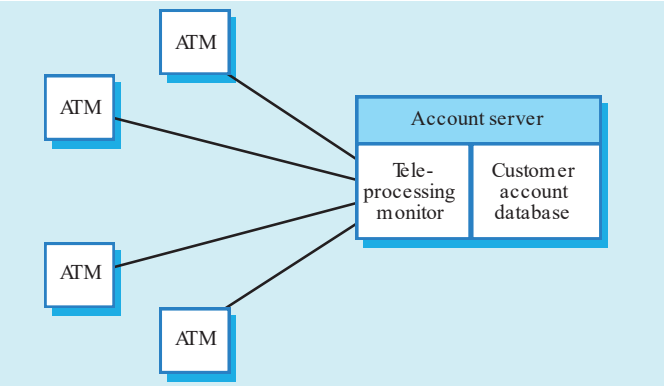
Thin client model

- Used when *legacy systems* are migrated to client/server architectures.
  - The legacy system acts as a server in its own right with a graphical interface implemented on a client.
- A major disadvantage is that it places a *heavy processing load on both the server and the network (performance & scalability issue)*.

Fat client model

- More processing is delegated to the client as the application processing is locally executed.
- Most suitable for new C/S systems where the capabilities of the client system are known in advance.
- More complex than a thin client model especially for management. *New versions of the application have to be installed on all clients (Maintainability issue)*.

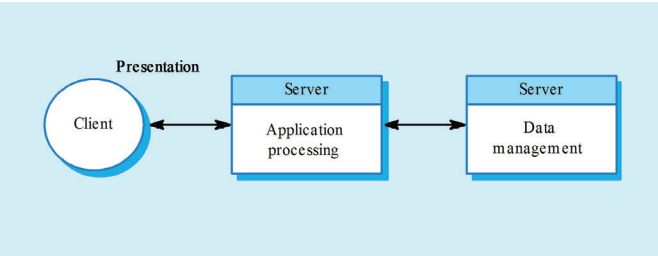
A client-server ATM system



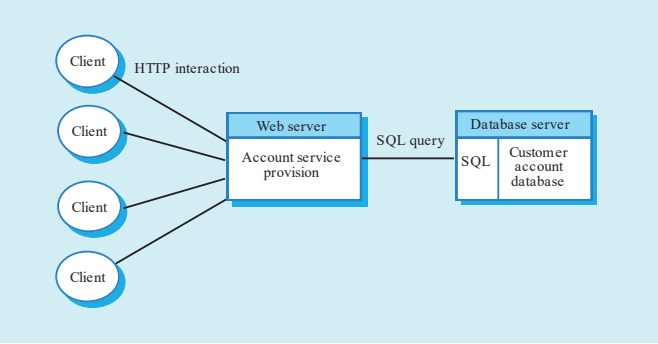
Three-tier architectures

- In a three-tier architecture, each of the application architecture *layers may execute on a separate processor*.
- Allows for *better performance and scalability than a thin-client approach and is simpler to manage than a fat-client approach*.
- A *more scalable architecture - as demands increase, extra servers can be added*.

A 3-tier C/S architecture



An internet banking system



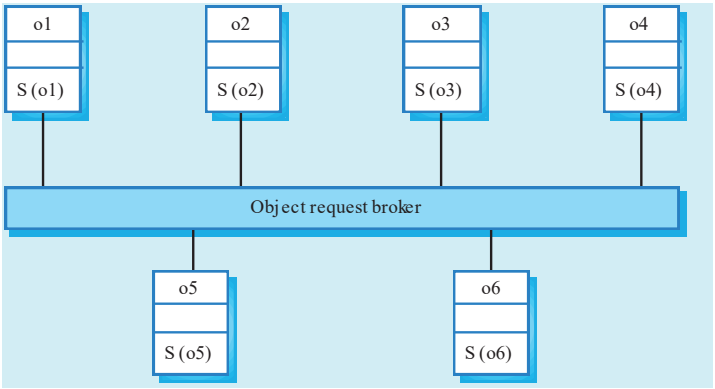
Use of C/S architectures

Architecture	Applications
Two-tier C/S architecture with thin clients	Legacy system applications where separating application processing and data management is impractical. Computationally-intensive applications such as compilers with little or no data management. Data-intensive applications (browsing and querying) with little or no application processing.
Two-tier C/S architecture with fat clients	Applications where application processing is provided by off-the-shelf software (e.g. Microsoft Excel) on the client. Applications where computationally-intensive processing of data (e.g. data visualisation) is required. Applications with relatively stable end-user functionality used in an environment with well-established system management.
Three-tier or multi-tier C/S architecture	Large scale applications with hundreds or thousands of clients Applications where both the data and the application are volatile. Applications where data from multiple sources are integrated.

Distributed object architectures

- There is ***no distinction*** in a distributed object architectures between clients and servers.
- Each distributable entity is *an object* that provides *services to other objects and receives services from other objects*.
- Object *communication is through a middleware* system called an *object request broker*.
- However, *distributed object architectures are more complex to design than C/S systems*.

Distributed object architecture



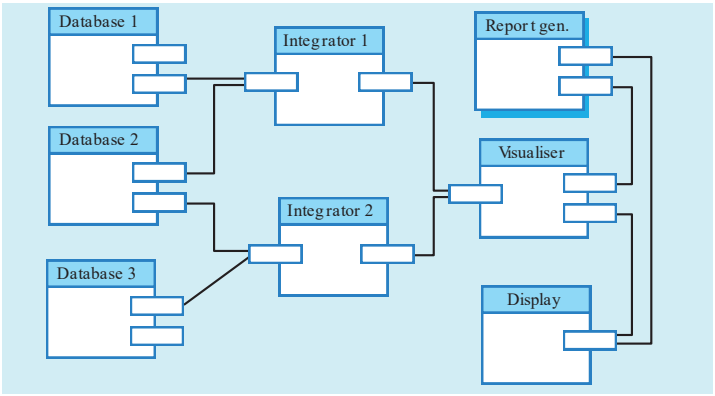
Advantages of distributed object architecture

- It allows the system designer to *delay decisions on where and how services should be provided*.
- It is a very *open system architecture* that allows new resources to be added to it as required.
- The system is *flexible and scaleable*.
- It is *possible to reconfigure the system dynamically* with objects migrating across the network as required.

Uses of distributed object architecture

- As a logical model allowing to *structure and organise the system*. In this case, you think about how to *provide application functionality solely in terms of services and combinations of services*.
- As a *flexible approach to the implementation of client-server systems*. The logical model of the system is a client-server model but both clients and servers are realised as distributed objects communicating through a common communication framework.

A data mining system



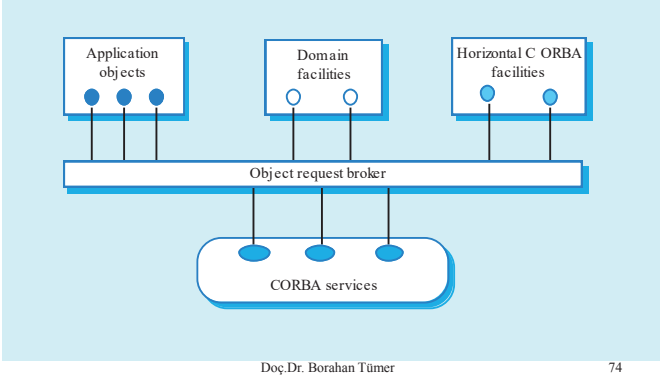
Data mining system

- The logical model of the system is not one of service provision where there are distinguished data management services.
- It allows the number of databases that are accessed to be increased without disrupting the system.
- It allows *new types of relationship* to be mined *by adding new integrator objects*.

CORBA

- CORBA: a *standard for an Object Request Broker - middleware to manage communications between distributed objects*.
- Middleware for distributed computing is required at 2 levels:
  - At the logical communication level, the middleware allows objects on different computers to exchange data and control information;
  - At the component level, the middleware provides a basis for developing compatible components. CORBA component standards have been defined.

CORBA application structure



CORBA Services

- Domain-independent interfaces used by many distributed object programs.
- Example: a service providing for the discovery of other available services; almost always necessary regardless of the application domain.
- The Naming Service -- which allows clients to find objects based on names; ***white pages***
- The Trading Service -- which allows clients to find objects based on their properties; ***yellow pages***

CORBA Common Facilities

- Application Objects: Objects developed for the specific application.
- Domain Facilities: *Standard objects defined for a specific domain*. The domain object standards cover *finance/insurance, e-commerce, health care*, ... etc.
- Fundamental Corba Services: provide basic *distributed computing services such as directories and security management.*

CORBA Common Facilities

- Horizontally-oriented (i.e., *common to many application domain*), but unlike object services they are oriented towards end-user applications.
- Example: *Distributed Document Component Facility* (DDCF), a compound document Common Facility based on OpenDoc.
- DDCF allows for presentation and interchange of objects based on a document model, for example, facilitating the linking of a spreadsheet object into a report document.

Peer-to-peer architectures

- Peer to peer (p2p) systems are *decentralised* systems where computations may be carried out by any node in the network.
- The overall system is designed to take advantage of the computational power and storage of a large number of networked computers.
- Most p2p systems have been personal systems but there is increasing business use of this technology.

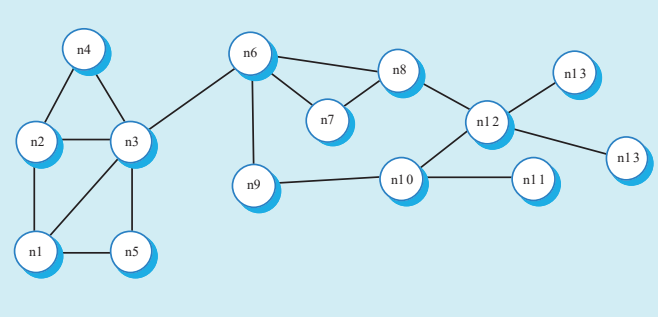
Examples to P2P architectures

- File sharing systems on PCs.
- Instant messaging systems such as ICQ to establish a direct communication between users.

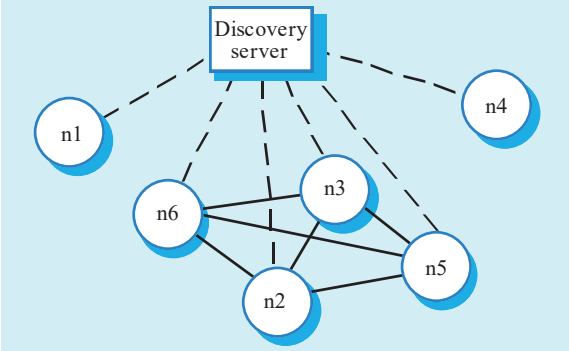
P2p architectural models

- The ***logical network*** architecture
  - *Decentralised* architectures;
  - *Semi-centralised* architectures.
- ***Application*** architecture
  - The generic organisation of components making up a p2p application.

Decentralised p2p architecture



Semi-centralised p2p architecture



Real Time (RT) or Embedded System Design

Embedded SW - 1

- Computers are used to *control* a wide range of systems including
  - simple domestic (in-house) machines,
  - games controllers, or
  - *entire* manufacturing plants.

Embedded SW - 2

- Their software must
  - *react to events* generated by the hardware and, often,
  - *issue control signals* in response to these events.

Embedded SW - 3

- SW in these systems is
  - *embedded in system hardware (mostly in ROM)*, and
  - usually *responds, in real time, to events from the system's environment*.

Responsiveness - 1

- *the critical difference* between RT or embedded and other software systems, such as
  - information systems,
  - web-based systems or
  - personal software systems

  is this real time reaction to these events.

Responsiveness - 2

- For *non-RT* systems, *correctness can be defined by specifying:*
  - how system inputs *map* to corresponding outputs that should be produced by the system.

Responsiveness - 3

- *For a RT system, correctness depends both on:*
  - *the response to an input (as for non-RT systems), **and***
  - *the time taken to generate that response*.
- *A response taking too late may be ineffective* and *the system incorrect !!!*

Definition - 1

- A *real-time system (RTS)* is some SW where the correct functionality depends on the extent to which it generates
  1. *expected outputs from the inputs presented, **and***
  2. *the time at which these results are produced*.

Definition - 2

- A *soft real-time system (soft RTS)* is a system
  - *whose operation is **degraded***
  - if results are not produced according to the specified timing requirements.

Definition - 3

- A *hard real-time system (hard RTS)* is a system
  - whose *operation is* ***incorrect***
  - if results are not produced according to the timing specification.

Characteristics of RT Systems

- generally *run continuously* and *do not terminate*.
- *Unpredictably interact* with the system's environment.
- *physical limitations may exist* that affect RTS design (e.g *military specifications*).
- *Direct hardware interaction* may be necessary (e.g. Hard reset or other calibration switches).
- *Issues of safety and reliability may dominate the system design* (*i.e., RTSs are critical systems !!!*)

RT Systems Design

- The design process for RTSs must *consider, in detail, the design and performance of the system hardware (a systems engineering process)*.
- Part of the design process may involve *deciding which system capabilities to implement in software and which in hardware.*
- *Low-level decisions on hardware, support software and system timing must be considered early in the process*.
- These may mean that *additional software functionality, such as battery and power management, has to be included in the system*.

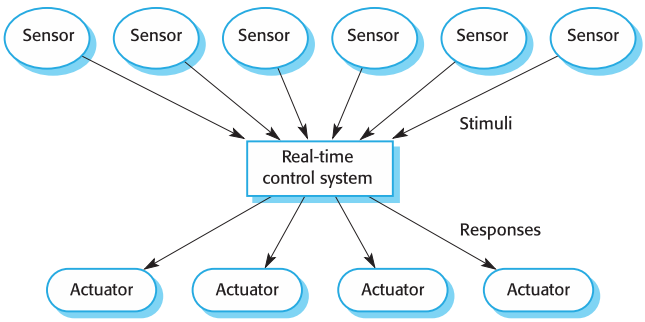
Reactive Systems

- *RTSs* are often considered to be *reactive systems*. Given a stimulus, the system must produce a reaction or response within a specified time.
- *Periodic stimuli*. Stimuli which *occur at predictable time intervals* (*e.g.*, a temperature sensor may be polled 10 times per second.)
- *Aperiodic stimuli*. Stimuli which *occur at unpredictable times* (*e.g.*, a system *power failure* may *trigger an interrupt which must be processed by the system*.

Stimuli and Responses for a Burglar Alarm System

Stimulus	Response
Clear alarms	*Switch off all active alarms; switch off all lights that have been switched on.*
Console panic button positive	*Initiate alarm; turn on lights around console; call police.*
Power supply failure	*Call service technician.*
Sensor failure	*Call service technician.*
Single sensor positive	*Initiate alarm; turn on lights around site of positive sensor.*
Two or more sensors positive	*Initiate alarm; turn on lights around sites of positive sensors; call police with location of suspected break-in.*
Voltage drop of between 10% and 20%	*Switch to battery backup; run power supply test.*
Voltage drop of more than 20%	*Switch to battery backup; initiate alarm; call police; run power supply test.*

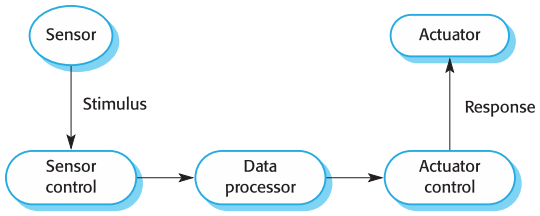
A General Model for an Embedded RTS



Architectural Considerations

- To *meet the timing demands* made by different stimuli/responses, the system *architecture must allow for fast switching between stimulus handlers*.
- *Timing demands of different stimuli are different* so a simple *sequential loop* is *not usually adequate*.
- *RTSs* are therefore usually *designed as cooperating processes* with *a real-time executive controlling* these processes.

Sensor and Actuator Processes



Architectural Considerations

- *Sensor control processes*
  - *Collect information* from sensors. *May buffer information collected* in response to a sensor stimulus.
- *Data processor*
  - *Carries out processing of collected information and computes the system response.*
- *Actuator control processes*
  - *Generate control signals for the actuators.*

Design process activities

- Platform selection
- Stimuli/response identification
- Timing analysis
- Process design
- Algorithm design
- Data design
- Process scheduling

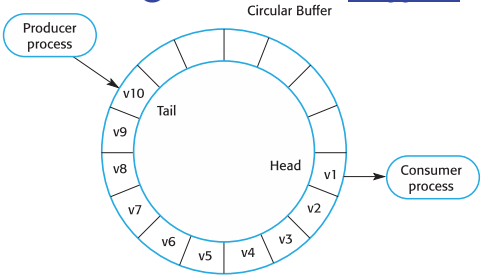
Process coordination

- *Processes* in a RTS have *to be coordinated and share information.*
- *Process coordination mechanisms ensure mutual exclusion to shared resources.*
- *Only one process is allowed to modify a shared resource at a given time*, other processes should wait.
- *When designing the information exchange between processes*, you have to *take into account* the fact that these *processes may be running at different speeds.*

Mutual exclusion

- *Producer (sensor control) processes collect data and add it to the **buffer**. Consumer (actuator control) processes take data from the **buffer** and make elements available.*
- *Producer and consumer processes must be mutually excluded from accessing the same element.*
- *The buffer must stop producer processes adding information to a full buffer and consumer processes trying to take information from an empty buffer.*

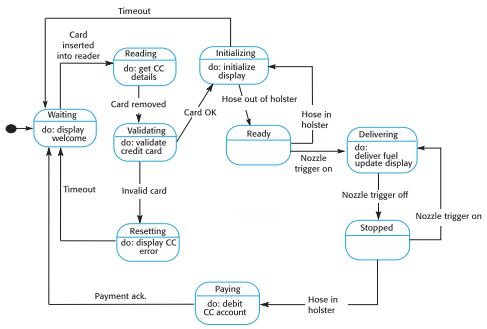
Producer/consumer processes sharing a circular ***buffer***



Real-time system modelling

- The *effect of a stimulus* in a RTS may trigger *a transition from one state to another.*
- *State models* are *therefore often used to describe embedded RTSs.*
- *UML state diagrams* may be used to *show the states and state transitions in a RTS.*

State machine model of a petrol (gas) pump



Sequence of actions in real-time pump control system

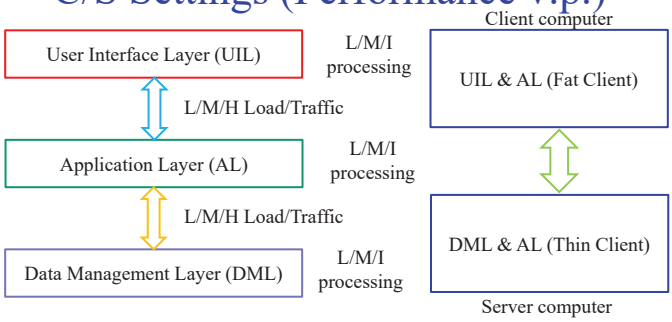
- The buyer *inserts a credit card* into a card reader built into the pump.
- *Removal of the card* triggers a *transition to a **validating** state where the card is validated.*
- *If the card is valid, the system initializes the pump* and, *when the fuel hose is removed from its holster, transitions to the **delivering** state.*
- After the fuel *delivery is complete* and the *hose replaced in its holster*, the *system moves to a **paying** state.*
- *After payment, the pump software returns to the **waiting** state*

Real-time programming

- *Programming languages for RTS development* have to include *facilities to access system hardware*. This *makes the prediction of the timing of particular operations possible* in these languages.
- *Systems-level languages*, such as ***C*** *allowing efficient code generation* are widely *used in preference to* languages like *Java*.
- There is a *performance overhead in object-oriented systems because extra code is required to mediate access to attributes and handle calls to operations*. The *loss of performance may make it impossible to meet real-time deadlines*.

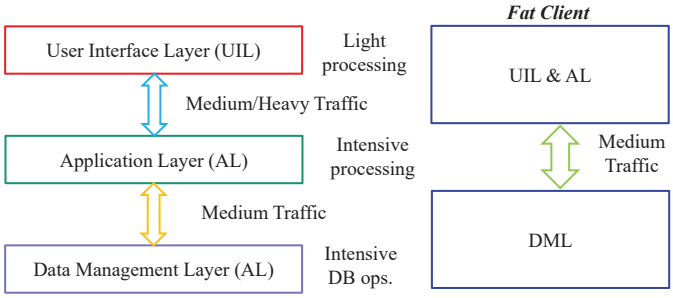
Supplementary Slides

Load/Processing Patterns versus C/S Settings (Performance v.p.)



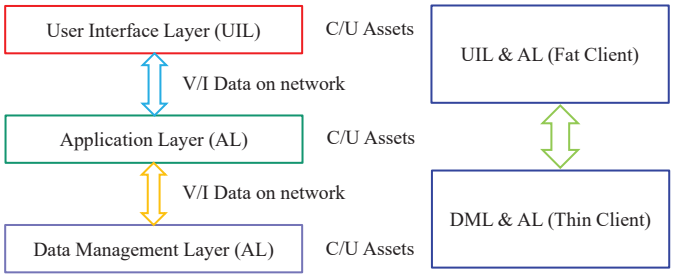
L: Light; M: Medium; H: Heavy; I: Intensive

Example Load/Processing Patterns for Fat Client Arch.



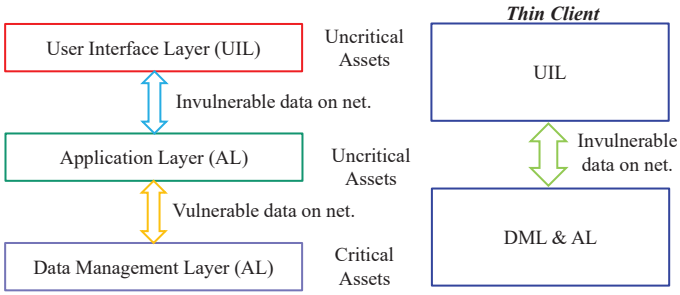
L: Light; M: Medium; H: Heavy; I: Intensive

Criticality/Vulnerability Patterns versus C/S Settings (Security vp).



C: Critical; U: Uncritical; V: Vulnerable; I: Invulnerable

Example Criticality/Vulnerability Patterns vs. for Thin Client Arch.



C: Critical; U: Uncritical; V: Vulnerable; I: Invulnerable